

Table of Contents

1. Introduction	3
1.1 Abstract	3
1.2 Problem Statement	4
1.3 Project Objectives	5
1.4 Literature Review - 3 Reviews	7
1.5 Project and System Scope	8
1.5.1 Project Scope	8
1.5.2 System Scope	14
1.5.2.1 Guest	14
1.5.2.2 User	18
1.5.2.3 Admin	25
2. System Overview	32
2.1 Generic Use Case	32
2.2 Class Diagrams	33
2.3 Entity Relationship Diagrams (ERD)	34
2.4 Object Diagrams	35
3. Functional and Non-Functional Requirements	36
3.1 Software Design Concepts	36
3.1.1 Abstraction	36
3.1.2 Encapsulation	38
3.1.3 Modularity	41
3.1.4 Functional Independence	43
3.1.5 Refinement	45
3.1.6 Refactoring	48
3.1.7 Architecture	50
3.1.8 Pattern	53
3.2 Design Principles	57
3.2.1 Single Responsibility Principle (SRP)	57
3.2.2 Open/Closed Principle (OCP)	60
3.2.3 Liskov Substitution Principle (LSP)	63
3.2.4 Interface Segregation Principle (ISP)	66
3.2.5 Dependency Inversion Principle (DIP)	69
4. Proposed Design Patterns	73
4.1 Proposed Design Pattern - Factory Method	73
4.1.1 Function or Software Component Affected	73
4.1.2 Description of work flow or data flow	74
4.1.3 Sample Class Diagram	76

4.1.4 Sample Other UML Notations (Sequence Diagram)	77
4.1.5 Sample Potential Code	78
4.1.6 Benefit	80
4.1.7 Limitations	81
4.2 Proposed Design Pattern - Observer	82
4.2.1 Function or Software Component Affected	82
4.2.2 Description of work flow or data flow	83
4.2.3 Sample Class Diagram	85
4.2.4 Sample Other UML Notations (ex State Diagram, WorkFlow Diagram)	86
4.2.5 Sample Potential Code	87
4.2.6 Benefit	89
4.2.7 Limitations	89
4.3 Proposed Design Pattern - Proxy	91
4.3.1 Function or Software Component Affected	91
4.3.2 Description of work flow or data flow	91
4.3.3 Sample Class Diagram	92
4.3.4 Sample Other UML Notations (ex State Diagram, WorkFlow Diagram)	93
4.3.5 Sample Potential Code	94
4.3.6 Benefit	95
4.3.7 Limitations	95
4.4 Proposed Design Pattern 4 - Strategy (Payment Functionality)	97
4.4.1 Function or Software Component Affected	97
4.4.2 Description of work flow or data flow	97
4.4.3 Sample Class Diagram	98
4.4.4 Sample Other UML Notations (ex State Diagram, WorkFlow Diagram)	100
4.4.5 Sample Potential Code	100
4.4.6 Benefit	103
4.4.7 Limitations	104
4.5 Proposed Design Pattern 5 - Facade - Login Functionality	105
4.5.1 Function or Software Component Affected	105
4.5.2 Description of Workflow or Data Flow	106
4.5.3 Sample Class Diagram	107
4.5.4 Sample Other UML Notations (sequence diagram)	109
4.5.5 Sample Potential Code	110
4.5.6 Benefits	111
4.5.7 Limitations	112
5. Conclusion and Suggestion	113
6. Bibilography/Reference	114

1. Introduction

1.1 Abstract

Welcome to Second Life, a dedicated online platform for buying and selling second-hand electronic devices. In today's fast-paced technological world, electronic products often become outdated quickly, despite remaining in excellent working condition. At Second Life, we aim to bridge this gap by offering a trusted and user-friendly portal that encourages the reuse of electronic items, promotes sustainability, and provides affordable options for users.

Our platform is designed with multiple user roles in mind – Admin, User, and Guest – each with specific capabilities to ensure smooth and secure transactions. As illustrated in our system use case diagram, Second Life offers comprehensive functionalities such as user registration, login, electronic item uploads, item browsing, search capabilities, order placement, payment processing, and shipment tracking. To ensure a safe marketplace, all uploaded items are subject to admin approval, and users can view or submit feedback based on their experiences.

Admins have powerful tools for managing user data, resolving complaints, handling reports, approving uploads, and overseeing order and electronic management. Registered users can enjoy a seamless shopping experience – from exploring products and making payments to viewing order details and interacting via a structured feedback system. Even guests can engage with our platform by browsing products and reading feedback before deciding to join our community.

Second Life is more than just a marketplace – it's a sustainable tech ecosystem. By giving electronic devices a second chance, we help reduce e-waste and support budget-conscious consumers in making smart, eco-friendly purchases. Whether you're a seller looking to pass on a gently used gadget or a buyer seeking affordable tech, Second Life is your go-to platform.

1.2 Problem Statement

Problem	Explanation
Problem 1: Lack of a Specialized Platform	Most online marketplaces are too general and not tailored for second-hand electronics. Buyers struggle to find specific items, and sellers can't provide detailed technical info like battery condition or model compatibility. This makes it harder to match the right buyer with the right product efficiently.
Problem 2: Trust Issues Between Buyers and Sellers	Users are often concerned about scams and the condition of the electronics. Without proper verification, reviews, or quality checks, buyers may receive faulty products, and sellers may deal with dishonest buyers. The lack of trust leads to fewer successful transactions.
Problem 3: Environmental Impact of Electronic Waste	Many people throw away old electronics due to the inconvenience of selling them. This contributes to e-waste, which harms the environment. A proper resale platform can reduce waste and help promote sustainability by encouraging device reuse.
Problem 4: Inconvenient User Experience	Existing platforms don't offer useful features for electronics, such as condition filters, battery info, or repair history. This makes searching and comparing listings time-consuming, especially for users unfamiliar with electronics.
Problem 5: No Centralized Feedback and Complaint System	There is often no easy way to report problems or share feedback after a transaction. Without a system to handle disputes or reviews, users can't voice concerns or warn others, which decreases platform credibility.
Problem 6: Lack of Admin Monitoring	Listings are rarely checked or approved by admins, leading to scams or fake products. Without proper monitoring, users are exposed to risks, and the platform can lose user trust quickly.

Table 1.2.1 problem statement

1.3 Project Objectives

1. Specialized Platform for Second-Hand Electronics

The system is developed to offer a dedicated marketplace focusing solely on used electronic products. By narrowing the platform's purpose, users can more easily find relevant listings, post electronic items for sale, and interact in a community that values device reuse. This specialization enhances the relevance, quality, and reliability of all transactions.

2. Build Trust Between Buyers and Sellers

The platform includes a verification system for sellers, product condition tags, and a feedback mechanism to promote transparency and trust. These features ensure that users can make informed decisions, reducing fraud and misunderstandings commonly found in general marketplaces.

3. Promote Environmental Sustainability

Second Life aims to reduce electronic waste by encouraging the resale and reuse of slightly used devices. By giving old electronics a second life, the system supports environmental protection efforts and promotes a circular economy model.

4. Admin Monitoring and Approval System

The platform includes an admin dashboard to monitor activities, approve product listings, resolve complaints, and generate reports. This ensures a safe, controlled, and high-quality experience for all users, reducing risks such as scams or illegal items.

5. Feedback and Complaint Resolution

A structured feedback and complaint system will allow users to review sellers, raise issues, and get assistance promptly. This builds user confidence and ensures fair dispute handling.

6. User-Friendly Interface and Experience

The system is designed with a clean, intuitive interface that allows users to easily browse, search, and filter electronics. It also supports real-time updates, payment options, and order tracking for a smoother transaction experience.

1.4 Literature Review - 3 Reviews

E-commerce	Description	Benefit	Evaluation
Mudah.my https://www.mudah.my	Mudah.my is a popular Malaysian online classified ads website where users can post and find second-hand items, including electronics.	- Large user base - Variety of product categories	Widely used but not specifically focused on electronics. Product condition and trust between users often vary.
Carousell https://www.carousell.com.my	Carousell is a C2C platform that allows users to buy and sell second-hand products. It is available in Malaysia and supports chat-based selling.	- User-friendly app - Direct chat with sellers	Has a large number of users, but the platform lacks strict seller verification, leading to some cases of scams.
Facebook Marketplace https://www.facebook.com/marketplace	Facebook Marketplace enables users to list and browse items in their area, including used electronics. Users contact each other via Messenger.	- Integrated with Facebook - Easy to reach local buyers and sellers	Convenient but not specialized. There is no official admin monitoring or buyer protection, which may cause reliability issues.

Table 1.4.1 literature review

1.5 Project and System Scope

1.5.1 Project Scope

Aspect	Details
Project Name	Online Second-Hand Electronic Selling Portal – Second Life
Objective	The Second Life platform is developed to provide a dedicated and trustworthy environment for buying and selling second-hand electronic products. It enables users to register, upload products, browse items, make purchases, and leave feedback securely. The system aims to reduce electronic waste by encouraging the reuse and resale of electronic goods. It features an intuitive user interface, smooth product management, secure transactions, and a feedback system to enhance trust among users. For administrators, it includes powerful tools for order approval, user management, report generation, and complaint resolution. The platform is scalable and adaptable for future integration with logistics, e-payment gateways, and eco-points reward systems.
Primary Users	Guests, Registered Users, and Admin
Key Feature	For Guests: 1. View Feedback

	Guests can read feedback from other users about products or sellers without logging in. This helps them evaluate seller reliability before deciding to buy. 2. Search and View Electronic Items Guests can browse and view second-hand electronics, including item details and user feedback, without creating an account. For Users: 1. Login Users can log in using their email and password to access personalized features. If login fails, they will see an error and can retry or use "Forgot Password". 2. Register New users can sign up by filling out a registration form with required info. Upon success, the system may auto-login or send a confirmation email. 3. Electronic Item Management Logged-in users can view, add, edit, or delete their second-hand electronic listings. All listings need admin approval before going live. 4. Search and View Items Users can search items by keywords or filters and view full details. If an item is sold or removed, they'll be notified. 5. Report Problem Users can report issues (e.g. fraud, fake listings) by submitting a
--	--

	form and optionally attaching photos. The system will notify admin. 6. Make Feedback After a transaction, users can rate and leave feedback for sellers. The feedback is saved and viewable by others. 7. View & Reply Feedback Sellers can view feedback left on their profiles and respond to comments to maintain good communication. 8. Payment Users confirm their orders and proceed with payment via various methods. Upon success, they'll receive confirmation and the order will be marked as "Paid". For Admin: 1. Login to Admin Panel Admins can securely log in to the admin dashboard using their credentials. This allows access to features like user management, item approval, and system monitoring not available to regular users. 2. Register New Admin Account Admins with the appropriate rights can create new admin accounts through the admin panel. This enables delegation of administrative tasks while maintaining role-based access control. 3. Approve Uploaded Items Admins review items uploaded by sellers to ensure they follow the website's policies. Approved items become
--	---

	visible to buyers, while rejected ones are returned to sellers with reasons. 4. Resolve Reported Issues Admins can address issues related to listed items, such as incorrect information or malfunction reports. Each issue is reviewed, resolved, and documented to maintain listing accuracy and integrity. 5. View System Reports Admins can access various system reports, including sales data, user activity logs, and inventory summaries. These reports support data-driven decision-making and performance monitoring. 6. Manage Users Admins can add, edit, deactivate, or remove users from the system. They can also assign roles and permissions to ensure secure and organized access based on responsibility levels.
Out of Scope	Future Implementation: As Suggestion 1. Logistics and Delivery Tracking System While users can complete orders and payments, delivery logistics and real-time tracking are not part of the current system. Examples: Delivery Scheduling, Order Tracking, Courier Management.

	2. Eco-Points Reward System The system does not yet implement any reward or point system to encourage eco-friendly behavior such as selling or buying second-hand items. Examples: Eco-Points Collection, Green Rewards, Sustainability Badges. 3. Mobile Application Support This version of the platform is web-based only and does not include dedicated mobile apps for Android or iOS. Examples: Push Notifications, In-App Purchases, Mobile-Optimized Features.												
Project Specification	Hardware Requirements for Owner and Customer <table border="1"> <thead> <tr> <th>Description</th><th>Requirements</th></tr> </thead> <tbody> <tr> <td>Operating System</td><td>Windows 10 or above</td></tr> <tr> <td>Processor</td><td>1.89 GHz or above</td></tr> <tr> <td>RAM</td><td>8 GB or above</td></tr> <tr> <td>Storage</td><td>16 GB free or above</td></tr> <tr> <td>Wireless Connection</td><td>Yes</td></tr> </tbody> </table> Software Requirements: ● Front End:Website Development using VS Code	Description	Requirements	Operating System	Windows 10 or above	Processor	1.89 GHz or above	RAM	8 GB or above	Storage	16 GB free or above	Wireless Connection	Yes
Description	Requirements												
Operating System	Windows 10 or above												
Processor	1.89 GHz or above												
RAM	8 GB or above												
Storage	16 GB free or above												
Wireless Connection	Yes												

	• Back End:Database management using phpMyAdmin and backend development with Flask • Operating System:Windows 8 / Windows 10 / Android 7.0 or above • Programming Languages:Python (Flask for backend development)
--	--

Table 1.5.1 project scope

1.5.2 System Scope

1.5.2.1 Guest

1.5.2.1.1 Login

Use Case Name	Login
Actors	Guest
Description	A guest can log in to their user account and access to the second-hand selling website using their credentials. The login process authenticates the user and grants access to personalized features and services.
Pre-Condition	The guest must have a registered account on the second-hand selling website.
Main Flow	1. The guest navigates to the second-hand selling website. 2. The guest locates the login section, typically found on the homepage or in the main menu. 3. The guest enters their registered email address into the designated field. 4. The guest enters their password into the designated field. 5. The guest submits their login credentials by clicking the "Login" button or pressing the "Enter" key. 6. The system verifies the guest's user credentials against the stored data in the database. 7. If the credentials are valid: The system logs the guest in and redirects them to their personalized dashboard or homepage. The system may display a welcome message with the user's name.
Post-Condition	The guest is successfully logged in to their user account and can access personalized features and services on the second-hand selling website.
Alternative Flow	If the credentials are invalid: The system displays an error message, such as "Invalid email or password." The guest may choose to retry logging in or use the "Forgot Password" feature.

Table 1.5.2.1.1 System Scope - Guest Login

1.5.2.1.2 Register

Use Case Name	Register
Actors	Guest
Description	A guest can create a new user account on the second-hand selling website. The registration process collects necessary information to set up an account and grants the guest access to the website's features and services.
Pre-Condition	The guest must have access to the internet and the second-hand selling website.
Main Flow	1. The prospective guest navigates to the second-hand selling website. 2. The guest locates the registration option, typically found on the homepage, login page, or in the main menu. 3. The guest clicks the "Sign Up" or "Create Account" button to begin the registration process. 4. The system displays the registration form, which may include fields for the guest's email address, username, password, and other personal information. 5. The guest fills out the registration form with the required information. 6. The guest agrees to the website's terms of service and privacy policy by checking the appropriate box. 7. The guest submits the registration form by clicking the "Submit" or "Create Account" button. 8. The system validates the guest's input to ensure all required fields are filled out correctly. 9. If the input is valid: -The system creates a new account with the provided information. -The system may send a confirmation email to the guest's registered email address. -The system logs the guest in automatically or redirects them to the homepage.
Post-Condition	The guest has successfully registered a user account and can access personalized features and services on the second-hand selling website.
Alternative Flow	If the input is invalid: -The system displays error messages indicating which fields need to be corrected. -The guest corrects the errors and resubmits the registration form.

Table 1.5.2.1.2 System Scope - Guest Register

1.5.2.1.3 View Feedback

Use Case Name	View Feedback
Actors	Guest
Description	A guest can view feedback left by other users , this can helps guests assess the credibility and reliability of sellers before making a purchase.
Pre-Condition	The guest must be visiting the website and have access to the internet. There is no requirement for the guest to be logged in to view feedback.
Main Flow	1. The guest navigates to the second-hand selling website. 2. The guest selects a product or a seller to view. 3. The system displays the product details or seller profile. 4. The guest chooses to view feedback. 5. The system retrieves and displays the feedback left by other users regarding the selected product or seller. 6. The guest reviews the feedback.
Post-Condition	The guest has successfully viewed the feedback and can make an informed decision based on the information provided by other users.
Alternative Flow	-

Table 1.5.2.1.3 System Scope - View Feedback

1.5.2.1.4 Search and View Electronic Items

Use Case Name	Search and View Electronic Items
Actors	Guest
Description	A guest can browse and view electronic items available for sale on the second-hand selling website. The guest can explore various categories, view item details, and read feedback from other users.
Pre-Condition	The guest must have access to the internet and the second-hand selling website.
Main Flow	1. The guest opens a web browser and navigates to the second-hand selling website. 2. The system displays a list of electronic items available for sale. 3. The guest browses through the list, viewing item images and brief descriptions. 4. The guest selects an item to view more details. 5. The system displays detailed information about the selected electronic item, including price, condition, specifications, and seller information. 6. The guest chooses to view feedback for the selected item. 7. The system retrieves and displays feedback left by other users regarding the selected electronic item. 8. The guest reviews the feedback and decides whether to proceed with a purchase or explore other items.
Post-Condition	The guest has successfully viewed and browsed electronic items and feedback, and can make an informed decision about potential purchases.
Alternative Flow	-

Table 1.5.2.1.4 System Scope - Search and View Electronic Items

1.5.2.2 User

1.5.2.2.1 Logout

Use Case Name	Logout
Actors	User
Description	A logged-in user can terminate their session securely from the system.
Pre-Condition	The user must be logged into the system.
Main Flow	1. The user clicks the “Logout” button or link on the navigation menu. 2. The system invalidates the current session and clears any session-related data. 3. The system redirects the user to the homepage or login page. 4. A confirmation message such as “You have successfully logged out” is displayed.
Post-Condition	1. The user is logged out of the system. 2. All session data is cleared to prevent unauthorized access.
Alternative Flow	-

Table 1.5.2.2.1 System Scope - User Logout

1.5.2.2.2 Electronic Item Management

Use Case Name	Electronic Item Management
Actors	User
Description	A user interacts with the system to manage electronic items, including viewing, adding, updating, and deleting item records.
Pre-Condition	The user is authenticated and logged into the system. The user has the appropriate role/permissions to manage items.
Main Flow	1. User logs into the Electronic Item Management System. 2. System authenticates and grants access to the dashboard. 3. User navigates to the Electronic Items section. 4. User performs one of the following actions: 5. View Items a. System displays a list of electronic items. b. User can filter/search items based on various criteria. 6. Add New Item a. User clicks "Add Item." b. User fills in item details (name, model, serial number, category, etc.). c. User submits the form. d. System validates and stores the item. e. After the admin approved item will be uploaded. 7. Update Existing Item a. User selects an item from the list. b. User edits the desired fields (e.g., quantity, warranty status). c. User saves changes. d. System updates the item. 8. Delete Item a. User selects an item. b. User clicks "Delete" and confirms the action. c. System removes or deactivates the item from inventory. 9. System confirms the action and updates the item list.
Post-Condition	The item list reflects the most recent changes. System logs the action for audit purposes. Any relevant notifications or stock alerts are updated.
Alternative Flow	-

Table 1.5.2.2.2 System Scope - Electronic Item Management

1.5.2.2.3 Search and View Electronic Item

Use Case Name	Search and View Electronic Item
Actors	User
Description	A user searches for second-hand electronic items using filters or keywords and views detailed information about selected items.
Pre-Condition	The user is authenticated and logged into the system. The user has the appropriate role/permissions to manage items.
Main Flow	1. User navigates to the search bar or category menu on the homepage. 2. User enters a search term (e.g., "laptop", "smartphone") or selects filters (e.g., category, price range, location). 3. System processes the query and displays a list of matching electronic items. 4. User browses the list and selects an item. 5. System displays the item's detail page, including title, images, description, price, seller info, and contact method. 6. User optionally saves the item, contacts the seller, or adds it to a wishlist (if logged in).
Post-Condition	The user has accessed the portal via a web browser or app. The portal's item database is available and connected.
Alternative Flow	If there is no matching result: - System informs the user that no items match the search criteria. - User is prompted to modify the search or reset filters. If the items is unavailable: - If the selected item has been removed or marked as sold, the system notifies the user and redirects to the previous results page.

Table 1.5.2.2.3 System Scope - Search and View Electronic Item

1.5.2.2.4 Report Problem

Use Case Name	Report Problem
Actors	User
Description	A user can file a complaint or report an issue related to an item, seller, buyer, or transaction (e.g., fraud, misleading information, non-delivery).
Pre-Condition	The user is authenticated and logged into the system.
Main Flow	1. User navigates to the “Report a Problem” section. 2. System will list a field to let the user file a complaint, and allow the user to upload a photo as evidence. 3. User inputs the problems and the user may upload a photo as proof. 4. User clicks the submit button. 5. System stores the complaint in the database and notifies the admin or support team. 6. System shows a confirmation message and optionally sends an email confirmation.
Post-Condition	1. Complaint is logged and assigned for review. 2. Admin or support team can access and respond to the report. 3. User may receive updates on complaint resolution.
Alternative Flow	-

Table 1.5.2.2.4 System Scope - Report Problem

1.5.2.2.5 Make a Feedback

Use Case Name	Make a Feedback
Actors	User
Description	A user can submit feedback about a seller, product, or platform experience after a transaction or interaction.
Pre-Condition	The user is authenticated and logged into the system.
Main Flow	1. User selects a seller selling section or profile. 2. System displays the option to leave feedback. 3. User fill in the rating and comment fields. 4. User submits the feedback. 5. System stores the feedback in the database. 6. System displays a confirmation message.
Post-Condition	1. Feedback is saved and linked to the seller. 2. Sellers can view the feedback.
Alternative Flow	-

Table 1.5.2.2.5 System Scope - Make a Feedback

1.5.2.2.6 View and Reply Feedback

Use Case Name	View and Reply Feedback
Actors	User
Description	A user, particularly a seller, to view feedback left by others and reply to it to maintain transparency and communication.
Pre-Condition	1. The user is authenticated and logged into the system. 2. Feedback exists for their product or profile.
Main Flow	1. User navigates to the “My Reviews” section. 2. System displays a list of received feedback. 3. User selects a specific feedback entry to view full details. 4. System provides a reply option. 5. User writes and submits a reply. 6. System saves the reply and links it with the feedback. 7. System shows a confirmation and updates the feedback view with the reply.
Post-Condition	Feedback is updated with the seller's reply.
Alternative Flow	-

Table 1.5.2.2.6 System Scope - View and Reply Feedback

1.5.2.2.7 Payment

Use Case Name	Payment
Actors	User
Description	A user completes a payment for a second-hand electronic item after confirming the order details.
Pre-Condition	1. The user is authenticated and logged into the system. 2. User has added an item to the cart and proceeded to checkout. 3. Order details (item, price, quantity, shipping) are confirmed.
Main Flow	1. User proceeds to the payment page after confirming the order. 2. System displays available payment options (e.g., credit/debit card, e-wallet, online banking). 3. User selects a payment method and enters required payment information. 4. System validates the payment information. 5. Payment is processed through the selected payment gateway. 6. On successful payment, the system updates the order status to "Paid". 7. System sends confirmation via email or in-app notification and redirects users to order summary.
Post-Condition	1. Payment is successfully recorded. 2. Order status is updated to reflect successful payment.
Alternative Flow	If the user cancels payment: - System aborts the payment and redirects the user back to the cart or order page. If invalid payment details: - System prompts users to correct information.

Table 1.5.2.2.7 System Scope - Payment

1.5.2.3 Admin

1.5.2.3.1 Login

Use Case Name	Login
Actors	Admin
Description	An administrator to log in to the admin panel of the second-hand selling website using their credentials. The admin panel provides access to advanced features and management tools that are not available to regular users.
Pre-Condition	The administrator must have a registered account with admin privileges on the second-hand selling website.
Main Flow	1. The administrator navigates to the second-hand selling website. 2. The administrator locates the admin login section, typically found in a hidden or protected area of the website. 3. The administrator enters their registered email address or username into the designated field. 4. The administrator enters their password into the designated field. 5. The administrator submits their login credentials by clicking the "Login" button or pressing the "Enter" key. 6. The system verifies the administrator's credentials against the stored data in the database. 7. If the credentials are valid and the account has admin privileges: The system logs the administrator in and redirects them to the admin dashboard. The system may display a welcome message with the administrator's name.
Post-Condition	The administrator is successfully logged in and can access the admin panel and its features.
Alternative Flow	If the credentials are invalid or the account does not have admin privileges: The system displays an error message, such as "Invalid email or password" or "Access denied." The administrator may choose to retry logging in or contact support for assistance.

Table 1.5.2.3.1 System Scope - Admin Login

1.5.2.3.2 Register

Use Case Name	Register
Actors	Admin
Description	An administrator to create a new administrator account on the second-hand selling website's admin panel. This process is critical to ensure that new administrators have the necessary permissions to manage the website.
Pre-Condition	An administrator must be logged in to the admin panel of the second-hand selling website.
Main Flow	1. The existing administrator navigates to the admin management section of the admin panel. 2. The existing administrator selects the option to add a new administrator. 3. The system displays a form for creating a new administrator account. 4. The existing administrator fills out the required information for the new administrator, including username, email address, and password. 5. The existing administrator sets the appropriate permissions and roles for the new administrator. 6. The existing administrator submits the form to create the new administrator account. 7. The system validates the input to ensure all required fields are filled out correctly and that the permissions are valid. 8. If the input is valid: The system creates the new administrator account with the provided information and permissions. The system may send a notification email to the new administrator with their login credentials and instructions.
Post-Condition	The new administrator account has been successfully created, and the new administrator can log in to the admin panel.
Alternative Flow	If the input is invalid: 1. The system displays error messages indicating which fields need to be corrected. 2. The existing administrator corrects the errors and resubmits the form.

Table 1.5.2.3.2 System Scope - Admin Register

1.5.2.3.3 Logout

Use Case Name	Logout
Actors	Admin
Description	A logged-in admin can terminate their session securely from the system.
Pre-Condition	The admin must be logged into the system.
Main Flow	1. The admin clicks the “Logout” button or link on the navigation menu. 2. The system invalidates the current session and clears any session-related data. 3. The system redirects the admin to the homepage or login page. 4. A confirmation message such as “You have successfully logged out” is displayed.
Post-Condition	1. The admin is logged out of the system. 2. All session data is cleared to prevent unauthorized access.
Alternative Flow	-

Table 1.5.2.3.3 System Scope - Admin Logout

1.5.2.3.4 Approval of Upload Item

Use Case Name	Approval of Upload Item
Actors	Admin
Description	An administrator reviewing and approving or rejecting items uploaded by sellers on the second-hand selling website. The approval process ensures that all listed items meet the website's guidelines and policies.
Pre-Condition	The seller must have uploaded an item for sale. The administrator must be logged in to the admin panel
Main Flow	1. The seller uploads an item for sale, providing all necessary details and images. 2. The system automatically sends the uploaded item for administrative review. 3. The administrator receives a notification about the new item awaiting approval. 4. The administrator navigates to the item approval section in the admin panel. 5. The administrator reviews the item details, including description, images, price, and seller information. 6. The administrator checks if the item complies with the website's guidelines and policies. 7. If the item is compliant: The administrator approves the item. The system updates the item status to "Approved" and makes it visible to buyers. The system may send a notification to the seller informing them of the approval.
Post-Condition	The uploaded item is either approved and listed for sale or rejected and removed from the listing queue.
Alternative Flow	If the item is not compliant: 1. The administrator rejects the item. 2. The system updates the item status to "Rejected" and provides a reason for rejection. 3. The system may send a notification to the seller informing them of the rejection and the reason. 4. The administrator logs the decision in the admin panel for future reference.

Table 1.5.2.3.4 System Scope - Approval of Upload Item

1.5.2.3.5 Problem Resolution

Use Case Name	Problem Resolution
Actors	Admin
Description	An admin resolves issues reported with electronic items, such as incorrect item data, missing entries, or malfunction reports.
Pre-Condition	Admin is logged into the system. A problem or issue has been reported The admin has full access rights to modify or manage item data.
Main Flow	1. Admin navigates to the Problem Reports or Issue Tracker section. 2. Admin views a list of reported problems related to electronic items. 3. Admin selects a specific issue to investigate. 4. System displays the details of the problem, including: - Item ID or reference - Problem description - Date/time reported - Reporting user (if applicable) 5. Admin reviews the item details and validates the issue by: - Checking the item data - Comparing with physical inventory (if needed) - Reviewing activity logs or change history 6. Admin takes corrective action based on the type of problem: - For incorrect data: Edits the item information. - For missing items: Adds the item or marks it as "unavailable." - For malfunction reports: Updates item status to "Faulty" or "Under Repair." - For duplicates: Merges or deletes redundant entries. 7. Admin adds a resolution note or comment to the problem log. 8. Admin marks the issue as Resolved.
Post-Condition	The problem is updated in the system with resolution status and notes. Affected item data is corrected or updated. The issue is no longer active or visible in the open issues list.
Alternative Flow	-

Table 1.5.2.3.5 System Scope - Problem Resolution

1.5.2.3.6 View Report

Use Case Name	View Report
Actors	Admin
Description	An admin to generate and view reports related to electronic items for decision-making, auditing, or inventory management purposes.
Pre-Condition	Admin is logged into the system. Report data is available Admin has reporting privileges.
Main Flow	1. Admin navigates to the Reports section of the system 2. System display report such as user activity log , purchase history and etc
Post-Condition	Admin successfully views the report.
Alternative Flow	-

Table 1.5.2.3.6 System Scope - View Report

1.5.2.3.7 User Management

Use Case Name	User Management
Actors	Admin
Description	Admin to manage users of the system, including creating, editing, deactivating, or assigning roles to users.
Pre-Condition	Admin is logged into the system. Admin has full access to user management features.
Main Flow	1. Admin navigates to the User Management section. 2. Admin performs one of the following actions: 3. A. Create New User Clicks “Add User” Fills in user details (name, email, username, role, password, department) Assigns a role (e.g., staff, supervisor, viewer) Submits the form System saves the user and sends login credentials (if applicable) 4. B. Edit User Searches for an existing user Selects the user from the list Updates relevant fields (e.g., email, role, status) Saves changes System updates user record 5. C. Deactivate/Delete User Selects a user Clicks “Deactivate” or “Delete” Confirms the action System deactivates or removes the user from active records 6. System logs all actions for auditing purposes.
Post-Condition	1. User accounts are created, updated, or deactivated accordingly. 2. Access control is enforced based on assigned roles. 3. All changes are recorded in the admin activity log.
Alternative Flow	-

Table 1.5.2.3.7 System Scope - User Management

2. System Overview

2.1 Generic Use Case



Figure 2.1 Use Case Diagram of Online Second-Hand Electronic Items Selling Portal

2.2 Class Diagrams

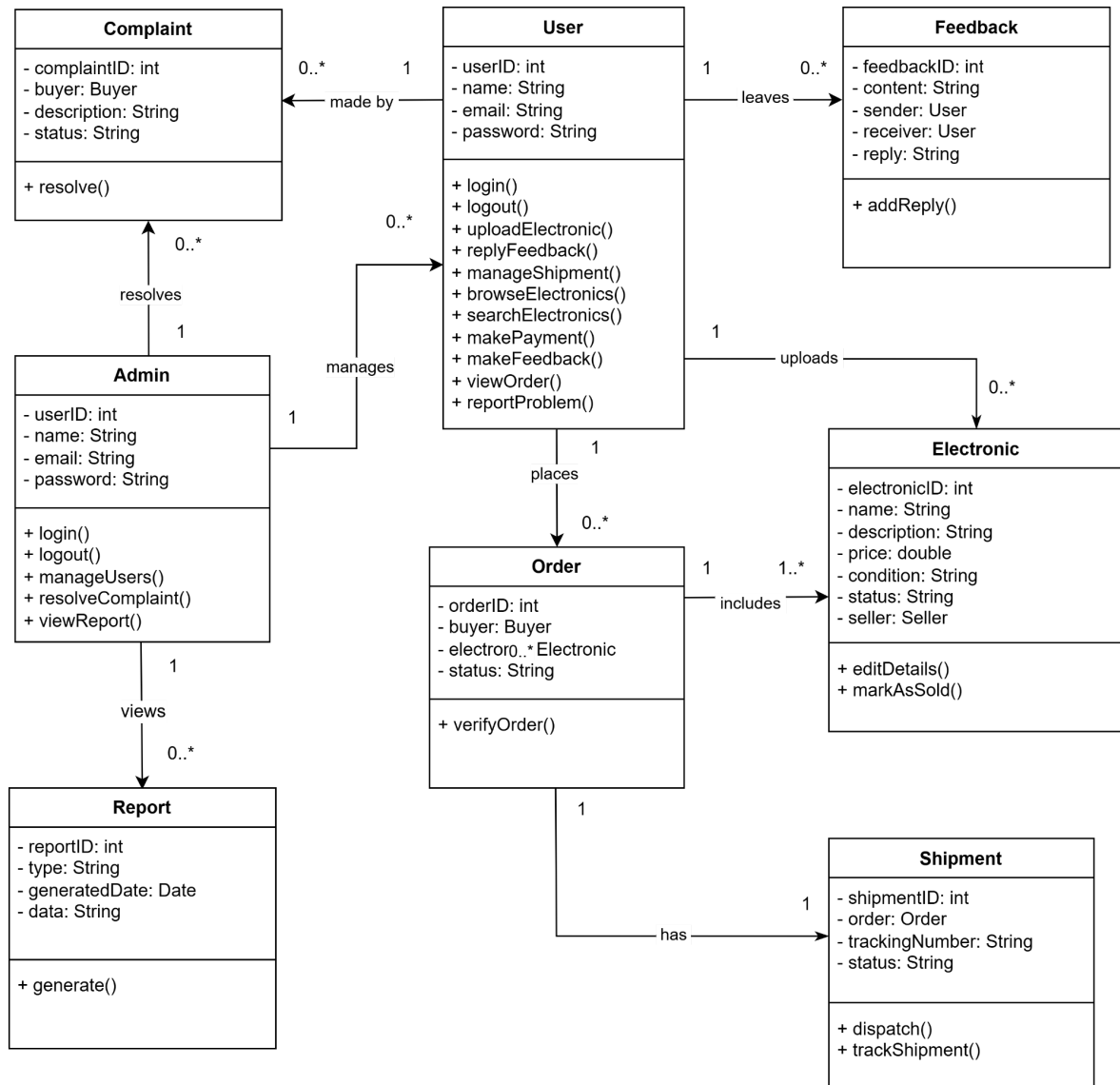


Figure 2.2 Class Diagram of Online Second-Hand Electronic Items Selling Portal

2.3 Entity Relationship Diagrams (ERD)

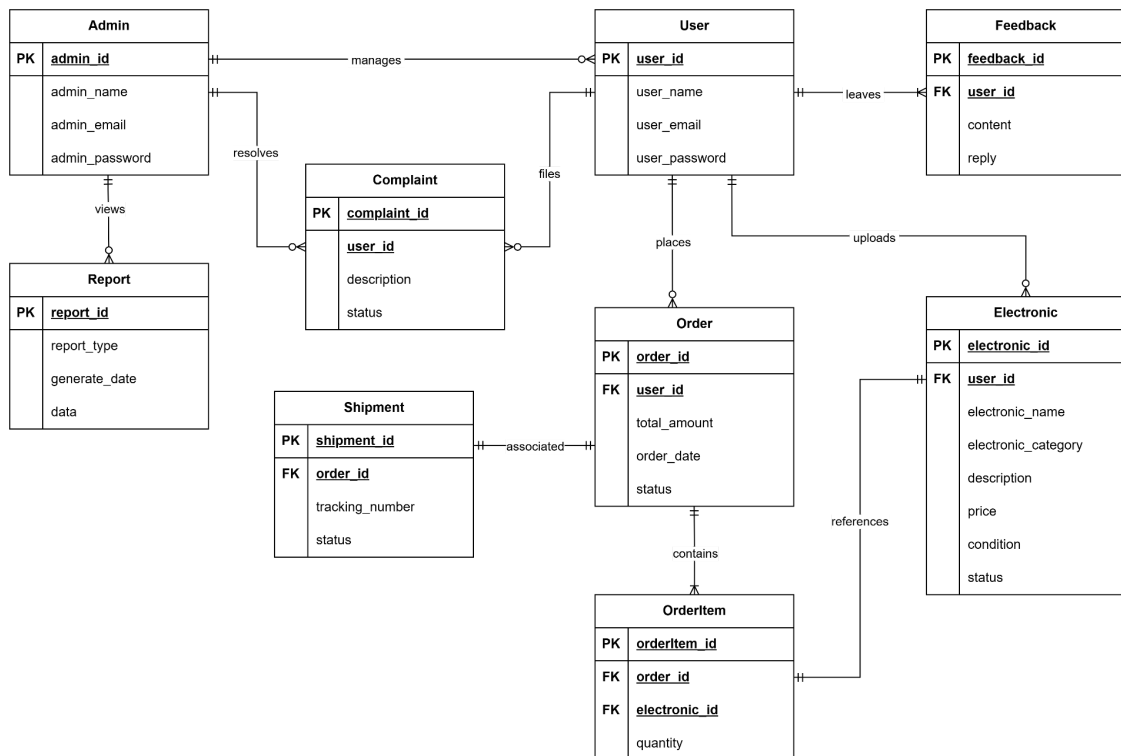


Figure 2.3 ERD of Online Second-Hand Electronic Items Selling Portal

2.4 Object Diagrams

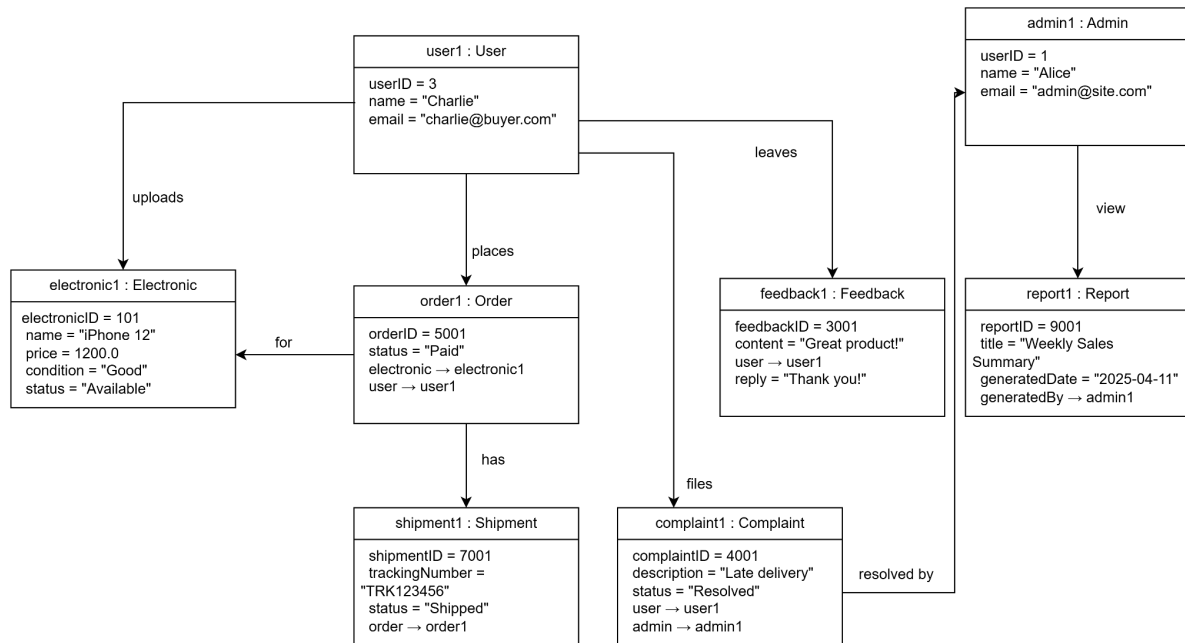


Figure 2.4 Object Diagram of Online Second-Hand Electronic Items Selling Portal

3. Functional and Non-Functional Requirements

3.1 Software Design Concepts

3.1.1 Abstraction

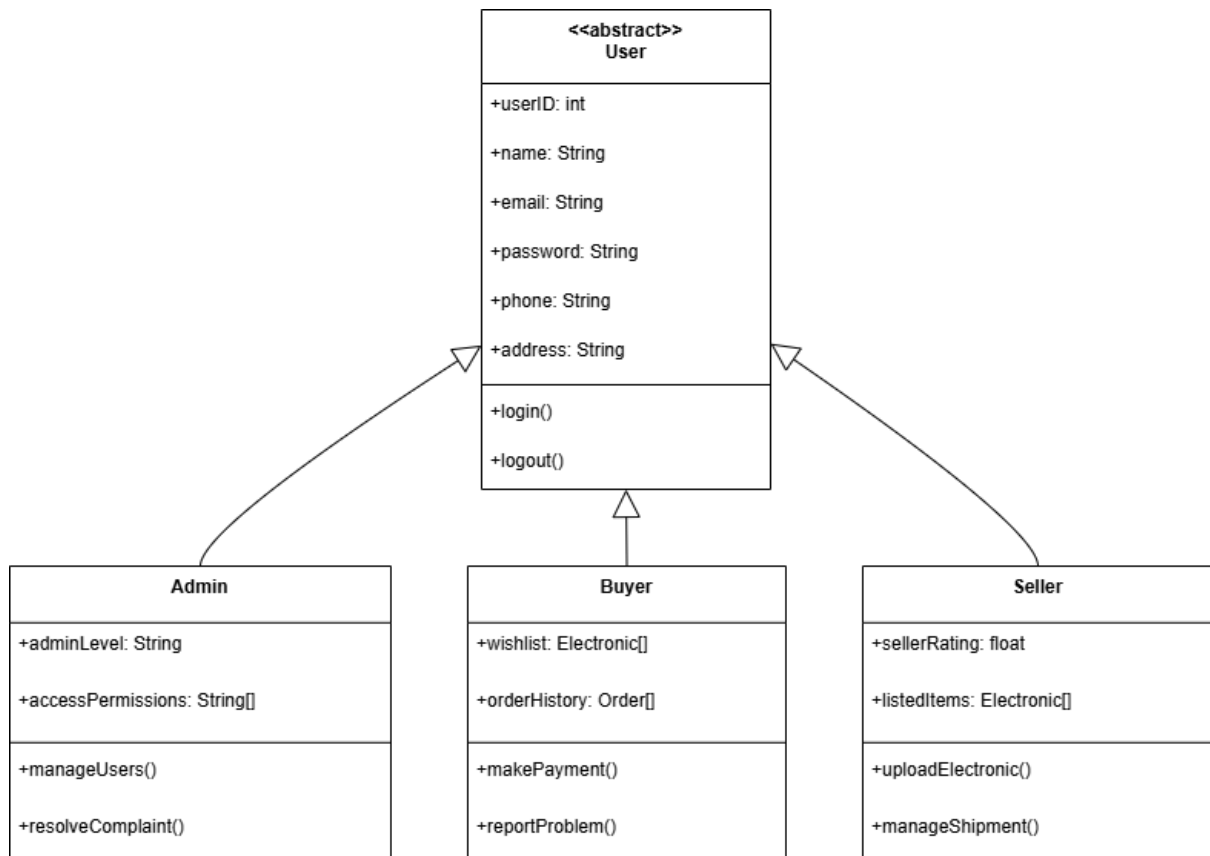


Figure 3.1.1 Abstraction

Abstraction simplifies complex systems by focusing on essential features while hiding implementation details. In SecondLife, the abstract User class defines core attributes (userID, name, email) and behaviors (login, logout) common to all user types, while specialized subclasses implement role-specific functionality:

- Buyer abstracts purchasing behavior with wishlist management and payment processing
- Seller abstracts inventory management with electronic listings and shipment tracking
- Admin abstracts system governance with user management and complaint resolution

This hierarchy allows the system to:

- Treat all users uniformly through the base User interface
- Extend functionality without modifying core user management
- Isolate role-specific details in subclasses

3.1.2 Encapsulation

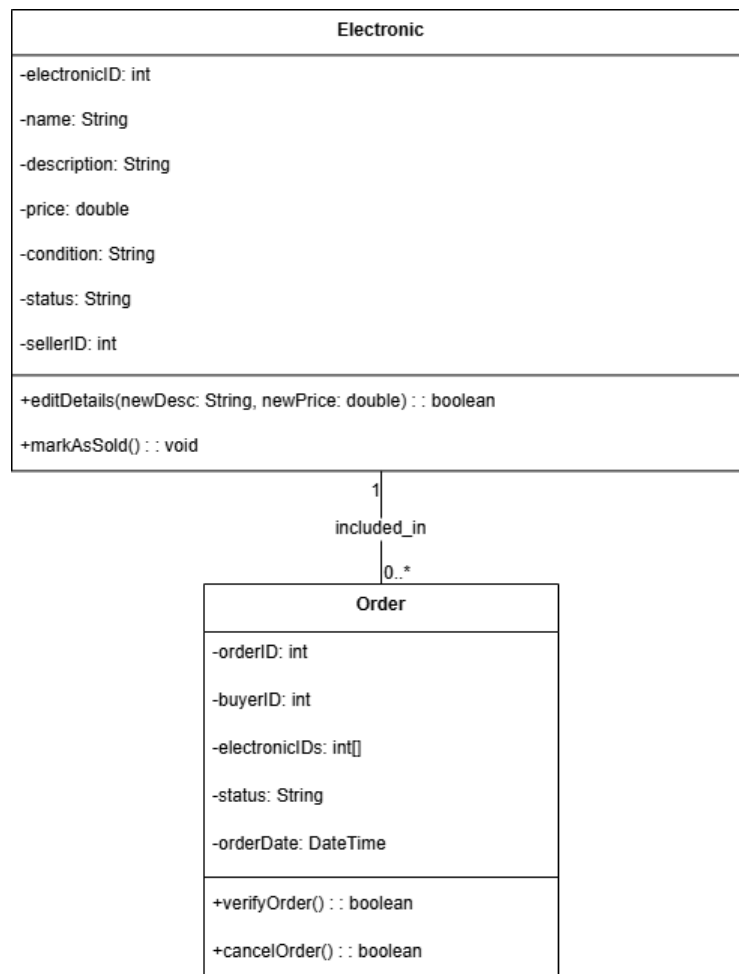


Figure 3.1.2 Encapsulation

Encapsulation is a fundamental principle that protects an object's internal state by restricting direct access to its data, exposing only controlled operations through public methods. In SecondLife, this is implemented through:

1. **Electronic Class Protection:**

- **Hidden Attributes:** Critical details like sellerID, condition, and status are declared private (- visibility)
- **Controlled Access:** Public methods (editDetails(), markAsSold()) enforce business rules:

```

class Electronic:
    def __init__(self):
        self._electronicID = None # Protected
        self.__sellerID = None    # Private
        self.__condition = None

    # Controlled access methods
    def edit_details(self, new_desc: str, new_price: float) ->
bool:
        if new_price <= 0:
            return False
        self.description = new_desc
        self.price = new_price
        return True

    @property
    def condition(self):
        return self.__condition

    @condition.setter
    def condition(self, value):
        if value in ["New", "Used", "Refurbished"]:
            self.__condition = value

```

Figure 3.1.2.1 Encapsulation Class Protection Code

2. Order Class Security:

- Status Management: The status attribute can only change through verifyOrder()/cancelOrder() with validation:

```

• class Order:
•     def __init__(self):
•         self.__status = "Pending"
•         self.__electronic_ids = []
•
•     def cancel_order(self) -> bool:
•         if self.__status == "Shipped":
•             return False
•         self.__status = "Cancelled"
•         return True
•
•     @property

```

```

•     def electronic_ids(self):
•         return self.__electronic_ids.copy() # Return a copy
      to prevent modification

```

Figure 3.1.2.1 Encapsulation Class Security Code

- Immutable Relationships: The electronicIDs array can't be modified externally after order creation

3. Relationship Enforcement:

- The 1..* multiplicity in Electronic -- Order ensures:
 - An electronic item can exist without being ordered (0..*)
 - All orders must reference at least one valid electronic item (1)

```

@app.route('/update_price/<int:electronic_id>', methods=['POST'])
@login_required
def update_price(electronic_id):
    if not current_user.is_seller: # Authorization check
        abort(403)

    electronic = Electronic.query.get_or_404(electronic_id)
    new_price = request.json.get('price')

    if not electronic.edit_details(new_price):
        return jsonify({"error": "Invalid price"}), 400

    db.session.commit()
    return jsonify({"message": "Price updated"})

```

Figure 3.1.2.1 Encapsulation Relationship Enforcement Code

Key Benefits for SecondLife:

- Fraud Prevention: Prevents unauthorized price/seller modifications
- Data Consistency: Guarantees valid order state transitions (e.g., can't mark shipped orders as "new")
- Audit Trail: All changes flow through methods that can log actions

- Maintenance Safety: Internal storage (e.g., condition grading system) can change without affecting callers

Real-World Analogy: Like a vending machine hiding its inventory mechanism while exposing only coin slots and product buttons, SecondLife's encapsulation ensures:

- Buyers interact only with product descriptions and prices
- Sellers manage listings through approved channels
- System integrity is maintained regardless of user actions

This design allows SecondLife to securely handle sensitive transaction data while providing flexibility for future enhancements (e.g., adding condition grading rules without API changes).

3.1.3 Modularity

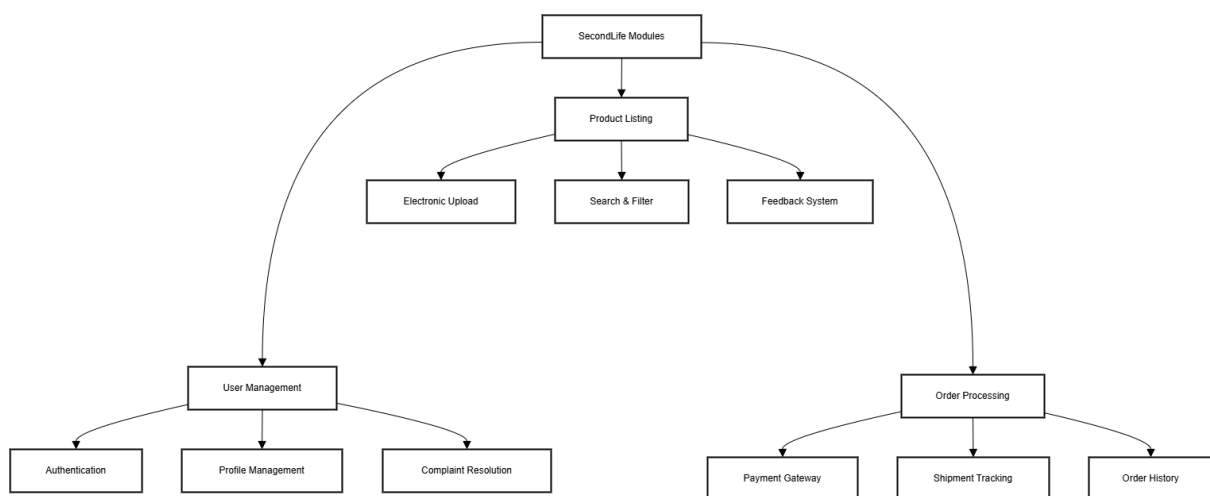


Figure 3.1.3 Modularity

Modularity organizes SecondLife into self-contained units that handle specific functions, communicating through clear interfaces.

Key Modules & Their Roles:

1. User Management

- Handles all user-related operations
- Subcomponents:
 - Authentication: Login/logout, session management
 - Profile Management: Personal data, preferences
 - Complaint Resolution: Dispute handling

2. Product Listing

- Manages electronics marketplace
- Subcomponents:
 - Electronic Upload: Seller product listings
 - Search & Filter: Buyer discovery tools
 - Feedback System: Ratings and reviews

3. Order Processing

- Manages transactions end-to-end
- Subcomponents:
 - Payment Gateway: Secure payment processing
 - Shipment Tracking: Delivery status updates
 - Order History: Purchase records

Implementation Advantages:

✓ Independent Operation

- Modules can be developed/tested separately (e.g., updating payment methods without affecting product search)

✓ Clear Responsibility Boundaries

- Each team owns specific modules (User, Product)

✓ Scalability

- High-traffic components (e.g., Payment Gateway) can be scaled independently

✓ Error Containment

- Issues in one module (e.g., Feedback System) don't crash others

✓ Technology Flexibility

- Modules can use different technologies (e.g., Redis for Authentication, Elasticsearch for Product Search)

Maintenance Benefits:

- Easier debugging (isolated issues)
- Simplified updates (change one module without rewriting others)
- Faster onboarding (new developers learn one module at a time)

This structure ensures SecondLife remains adaptable as it grows into additional modules without disrupting existing systems.

3.1.4 Functional Independence

Feedback	Shipment	Report
-feedbackID: int -content: String -rating: int -timestamp: DateTime	-shipmentID: int -carrier: String -trackingNumber: String -dispatchDate: DateTime	-reportID: int -type: String -generationDate: DateTime -format: String
+addReply() +editContent()	+trackShipment() +updateStatus()	+generate() +export()

Figure 3.1.4 Functional Independence

Functional Independence ensures that each module or class in SecondLife performs a single, well-defined task with minimal reliance on other components. This is measured by high cohesion (focused functionality) and low coupling (minimal dependencies).

Key Characteristics in SecondLife:

1. High Cohesion (Focused Responsibility)

Each class handles a specific, narrow task:

- Feedback Class
 - Manages user reviews and replies
 - Contains only feedback-related attributes (content, rating, timestamp)
 - Exposes only feedback-specific methods (addReply(), editContent())
- Shipment Class
 - Solely handles delivery tracking
 - Attributes (carrier, trackingNumber) and methods (trackShipment()) are strictly shipment-related
- Report Class
 - Dedicated to report generation
 - No mixed logic (e.g., doesn't handle payments or user profiles)

2. Low Coupling (Minimal Dependencies)

- No direct dependencies between Feedback, Shipment, and Report.
- They interact through well-defined interfaces (e.g., an Order class might use Shipment, but Shipment doesn't need to know about Feedback).
- Changes to one class (e.g., modifying Report formats) don't ripple to others.

Benefits of Functional Independence in SecondLife

✓ Easier Maintenance

- Bugs in Feedback won't break Shipment or Report.

✓ Reusability

- Shipment can be reused in other projects (e.g., a logistics system) without dragging in unrelated code.

✓ Scalability

- New features (e.g., a Refund class) can be added without modifying existing independent modules.

✓ Team Parallelism

- Developers can work on Feedback and Report simultaneously without conflicts.

3.1.5 Refinement

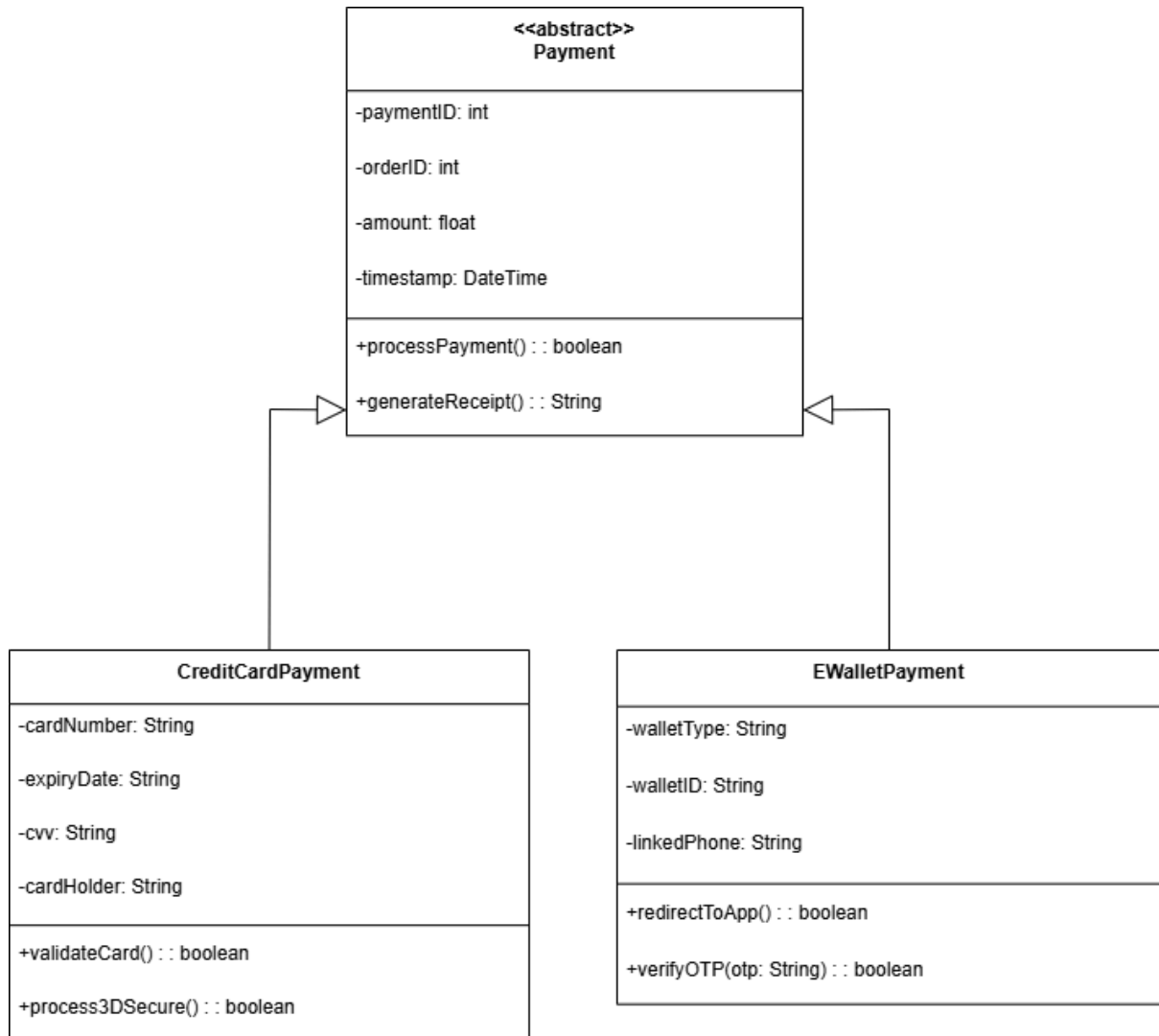


Figure 3.1.5 Refinement

Refinement is the process of breaking down a general, high-level concept into more specialized and detailed subcomponents. In SecondLife, this is demonstrated through the Electronic class hierarchy, where a broad Electronic class is refined into specific subclasses (UsedElectronic and RefurbishedElectronic) to handle distinct types of second-hand electronics.

How Refinement Works in SecondLife

1. Base Class (Electronic – Generalization)

- Defines core attributes common to all electronics:
 - electronicID, model, brand, specs
- Provides a generic method (listPrice()) applicable to all electronics.
- Serves as a template for specialized subclasses.

2. Refined Subclasses (Specialization)

Each subclass adds unique attributes and behaviors:

- UsedElectronic
 - Focuses on pre-owned items with:
 - usageDuration
 - physicalCondition
 - originalPurchaseDate (for authenticity checks)
 - May override listPrice() to factor in wear-and-tear discounts.
- RefurbishedElectronic
 - Handles professionally restored items with:
 - refurbisherCert
 - warrantyPeriod
 - May include additional methods like verifyCertification().

Benefits of Refinement in SecondLife

✓ Clear Specialization

- Buyers instantly recognize differences between used vs. refurbished items.
- Sellers provide precise details without cluttering the base class.

✓ Flexible Extensions

- New subtypes (e.g., VintageElectronic) can be added without modifying the base Electronic class.

✓ Consistent Interface

- All electronics share core attributes (brand, model), ensuring uniformity in listings.
- Subclasses extend rather than replace base functionality (e.g., `listPrice()` works for all types).

✓ Maintainability

- Changes to `UsedElectronic` (e.g., adding `batteryHealth`) don't affect `RefurbishedElectronic`.

Importance

Refinement mirrors real-world categorization:

- Transparency: Buyers know exactly what they're getting.
- Efficiency: Sellers don't waste time entering irrelevant details (e.g., warranty for non-refurbished items).
- Scalability: New product types (e.g., "For Parts") integrate seamlessly.

By refining the `Electronic` class, `SecondLife` achieves a structured, scalable, and user-friendly system for second-hand electronics trading.

3.1.6 Refactoring

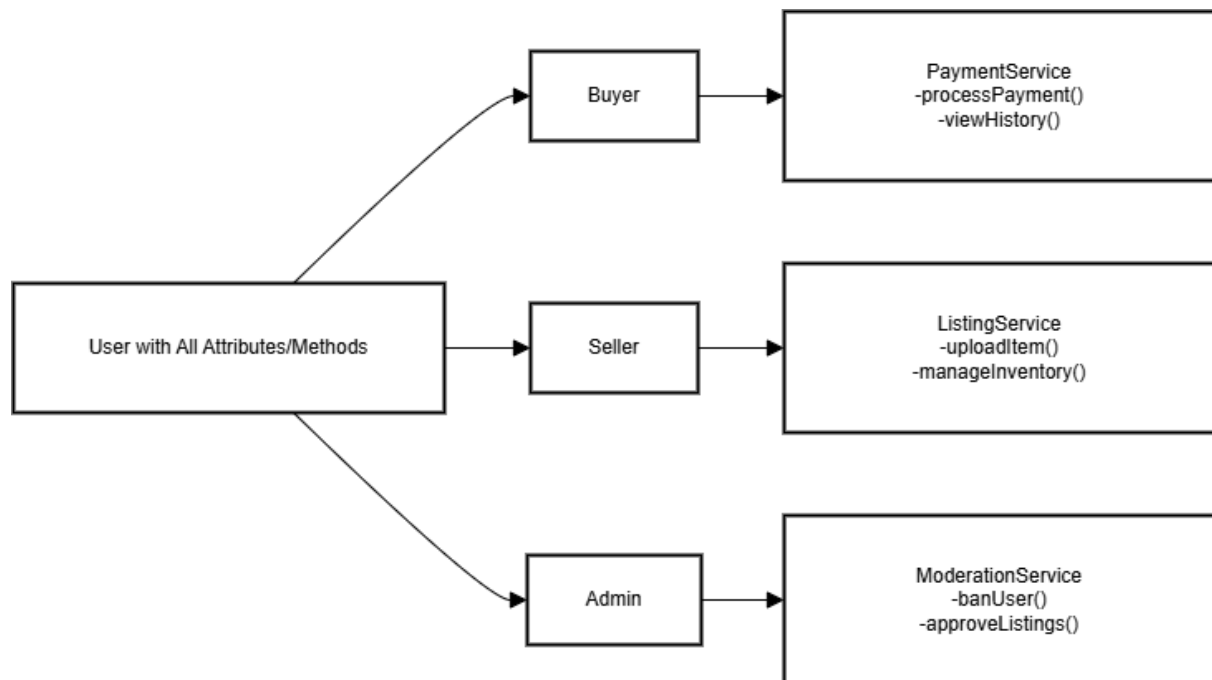


Figure 3.1.6 Refactoring

Refactoring improves code structure without changing functionality, making it more maintainable and scalable. In SecondLife, the User class was initially monolithic—handling all roles (Buyer, Seller, Admin) in one place, leading to bloated and hard-to-maintain code.

Before Refactoring (Problem)

- A single User class contained:
 - Buyer logic (payments, wishlists)
 - Seller logic (inventory, shipments)
 - Admin logic (banning users, approving listings)
- Issues:
 - High complexity: A change in payment logic risked breaking admin features.
 - Poor readability: Difficult to distinguish role-specific behaviors.
 - Low reusability: Buyer logic couldn't be reused in other projects.

After Refactoring (Solution)

The system was split into:

1. Role-Specific Classes (Buyer, Seller, Admin)
 - Each handles only its own responsibilities.
 - Example: Buyer manages payments, while Seller focuses on listings.
2. Dedicated Services (Extracted from the original User)
 - PaymentService: Handles transactions separately from user management.
 - ListingService: Manages product uploads independently.
 - ModerationService: Isolates admin-only actions like banning users.

Why This Matters

- Simpler Maintenance: Fixing a payment bug doesn't require sifting through unrelated seller/admin code.
- Easier Scaling: New features (e.g., a VIP Buyer subclass) can be added cleanly.
- Better Teamwork: Developers can work on separate services without conflicts.

This refactoring mirrors real-world organization—like splitting a general "store employee" into specialized roles (cashier, stocker, manager) for efficiency. SecondLife's code is now cleaner, safer, and ready for growth.

3.1.7 Architecture

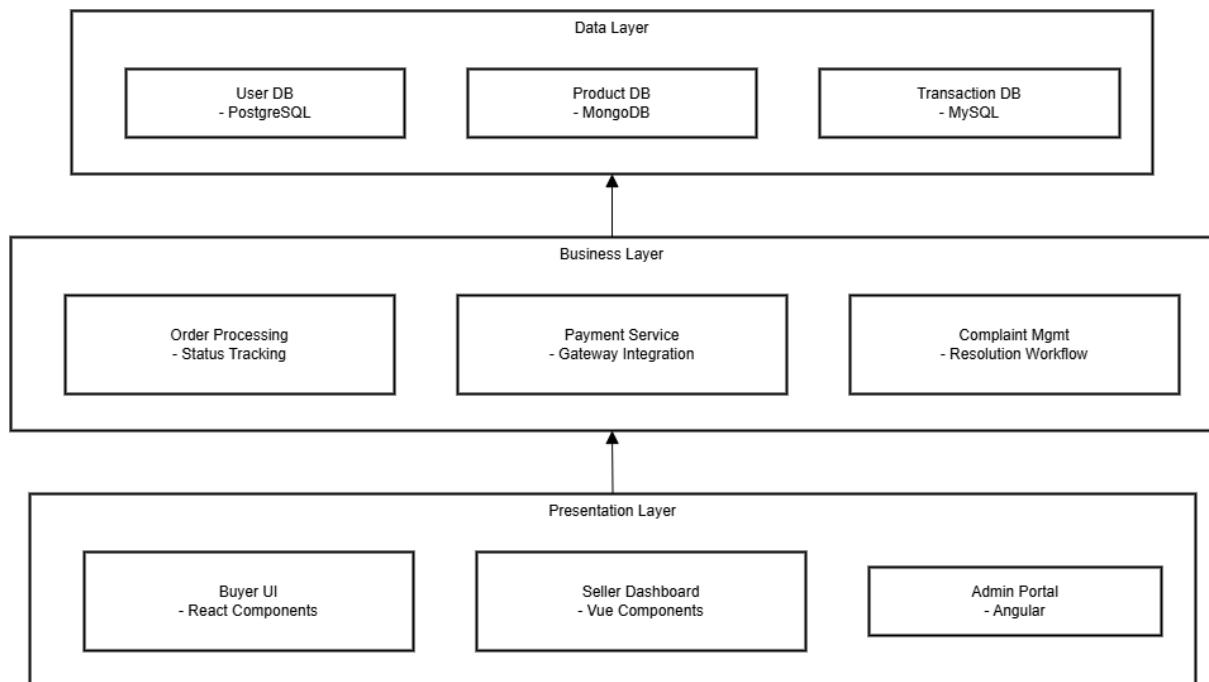


Figure 3.1.7 Architecture

The software architecture of SecondLife follows a layered (n-tier) structure, ensuring separation of concerns, scalability, and maintainability. Built with Python (Flask) for the backend, HTML/CSS/JavaScript for the frontend, and multiple databases, the system is organized into three primary layers:

1. Presentation Layer (Frontend – User Interfaces)

Technology: HTML/CSS, JavaScript, Flask Templates

Components:

- Buyer UI
 - Handles browsing electronics, cart management, and order placement.
- Seller Dashboard
 - Manages product listings, order fulfillment, and shipment updates.
- Admin Portal

- Provides tools for user moderation, report generation, and system analytics.

Key Role:

- Serves as the user-facing interface, rendering data from the backend.
- Example: When a buyer clicks "Add to Cart", the frontend sends an HTTP request to the Business Layer.

2. Business Layer (Backend Logic – Flask/Python)

Technology: Python (Flask), textfile

Components:

- Order Processing
 - Manages order status (e.g., "Confirmed" → "Shipped") and validates transactions.
- Payment Service
 - Integrates with payment gateways via Flask routes.
- Complaint Management
 - Resolves disputes between buyers/sellers using workflow logic.

Key Role:

- Contains core business rules (e.g., "Sellers can't cancel shipped orders").
- Processes requests from the Presentation Layer and interacts with the Data Layer.
- Example:

```

● @app.route('/api/orders/<int:order_id>/ship',
  methods=['POST'])
● def ship_order(order_id):
●     order = Order.query.get_or_404(order_id)
●     if order.status != "Confirmed":

```

```

●         return jsonify({"error": "Order not ready for
shipping"}), 400
●         order.status = "Shipped"
●         db.session.commit()
●         return jsonify({"message": "Order shipped
successfully"})

```

Figure 3.1.7.1 Architecture Business Layer Code

3. Data Layer (Database & Storage)

Components:

- **User DB**
 - Stores buyer/seller/admin profiles, login credentials, and addresses.
- **Product DB**
 - Flexible schema for electronics listings (e.g., dynamic specs for laptops vs. smartphones).
- **Transaction DB**
 - ACID-compliant order/payment records for audit trails.

Key Role:

- Provides **persistent storage** with optimized queries
 - Ensures data consistency

Key Advantages for SecondLife

✓ Separation of Concerns

- Frontend developers work independently of backend logic.
- Database schema changes don't break UI components.

✓ Scalability

- Each layer can scale horizontally (e.g., adding more servers for payment processing).

✓ Security

- Sensitive operations (e.g., payments) are isolated in the Business Layer.
- Database credentials are never exposed to the frontend.

✓ Technology Flexibility

- The frontend can switch from React to Vue without altering backend logic.
- MongoDB's flexibility accommodates diverse electronic product specs.

This architecture ensures SecondLife remains **robust, adaptable, and efficient** as it grows.

3.1.8 Pattern

1. Factory Method Pattern

Definition:

Provides an interface for creating objects while allowing subclasses to decide which class to instantiate.

Application in SecondLife:

- The Product Listing Module uses the Factory Method Pattern to create different types of electronic products (Used, Refurbished, New) based on seller input.
- When a seller lists an item, the `ElectronicFactory` determines the appropriate product subclass to instantiate based on the condition selected.

Example Implementation:

```
class ElectronicFactory:

    def create_electronic(self, condition, details):

        if condition == "used":

            return UsedElectronic(details)

        elif condition == "refurbished":

            return RefurbishedElectronic(details)
```

```

        elif condition == "new":

            return NewElectronic(details)

# Usage

factory = ElectronicFactory()

new_listing = factory.create_electronic("refurbished", {...})

```

Figure 3.1.8.1 Factory Method Pattern Code

2. Observer Pattern

Definition:

Establishes a one-to-many dependency between objects so that when one object changes state, all its dependents are notified.

Application in SecondLife:

- The Order Management System uses this pattern to notify buyers and sellers about order status changes.
- Both users and the delivery system can subscribe to order updates.

Example Implementation:

```

class Order:
    def __init__(self):
        self._observers = []
        self._status = "Pending"

    def attach(self, observer):
        self._observers.append(observer)

    def notify(self):
        for observer in self._observers:
            observer.update(self)

class BuyerNotification:
    def update(self, order):
        send_email(order.user.email, f"Order {order.id} is now {order.status}")

```

Figure 3.1.8.2 Observer Pattern Code

3. Proxy Pattern

Definition:

Provides a surrogate or placeholder for another object to control access to it.

Application in SecondLife:

- The Product Display System uses proxy objects for product images to optimize loading.
- High-resolution images are only fetched when explicitly requested by the user.

Example Implementation:

```
class ProductImageProxy:
    def __init__(self, image_id):
        self.image_id = image_id
        self._real_image = None

    def display(self):
        if self._real_image is None:
            self._real_image = HighResImageLoader.load(self.image_id)
        return self._real_image.display()
```

Figure 3.1.8.3 Proxy Pattern Code

4. Strategy Pattern

Definition:

Defines a family of algorithms, encapsulates each one, and makes them interchangeable.

Application in SecondLife:

- The Payment Processing System implements different payment methods as interchangeable strategies.
- Each payment option (Credit Card, TNG, GrabPay) has its own strategy class.

Example Implementation:

```
class PaymentStrategy(ABC):
    @abstractmethod
    def process(self, amount):
        pass

class CreditCardStrategy(PaymentStrategy):
    def process(self, amount):
        # Process credit card payment
        return "Credit card payment processed"
```

```
class PaymentProcessor:
    def __init__(self, strategy):
        self._strategy = strategy

    def execute_payment(self, amount):
        return self._strategy.process(amount)
```

Figure 3.1.8.4 Strategy Pattern Code

5. Facade Pattern

Definition:

Provides a unified interface to a set of interfaces in a subsystem.

Application in SecondLife:

- The Order Placement System provides a simplified facade that hides the complexity of:
 - Inventory checks
 - Payment processing
 - Shipment scheduling
 - Notification sending

Example Implementation:

```
class OrderFacade:
    def place_order(self, user_id, product_ids):
        # Simplified interface hiding complex operations
        if not Inventory.check_availability(product_ids):
            raise Exception("Items not available")

        total = Calculator.get_total(product_ids)
        PaymentProcessor.process(user_id, total)

        order = Order.create(user_id, product_ids)
        NotificationService.notify(order)

        return order
```

Figure 3.1.8.5 Facade Pattern Code

Each pattern addresses specific challenges in SecondLife while working together to create a cohesive, maintainable architecture that can evolve with the platform's growth. The patterns are implemented consistently across the Python/Flask backend and JavaScript frontend to ensure seamless operation.

3.2 Design Principles

3.2.1 Single Responsibility Principle (SRP)

The Single Responsibility Principle (SRP) states that every class or module in a software system should have only one reason to change, meaning it should encapsulate a single responsibility or functionality. By adhering to SRP, each component becomes more focused, maintainable, and reusable, as changes to one part of the system (e.g., authentication logic) don't risk breaking unrelated features (e.g., profile management). For example, in SecondLife, separating AuthService (login/logout), ProfileService (user data), and OrderService (order processing) ensures that modifying payment logic won't impact user authentication, reducing complexity and improving scalability. This principle is foundational to solid design, promoting modular, robust, and adaptable code.

Before SRP

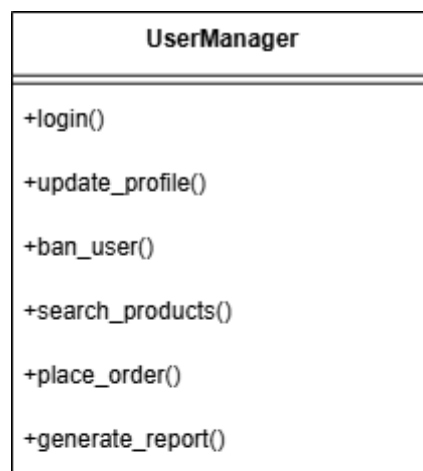


Figure 3.2.1.1 Before Single Responsibility Principle (SRP)

```
class UserManager:
    def login(self, username, password): ...
    def update_profile(self, name, address): ...
    def ban_user(self, user_id): ... # Admin logic mixed in
    def search_products(self, query): ... # Product logic mixed in
```

Figure 3.2.1.2 Code Before Single Responsibility Principle (SRP)

Issues:

- Tight coupling: Changes to `ban_user()` might break `login()`.
- Low cohesion: Unrelated methods in one class.

In the initial design, the `UserManager` class violates SRP by handling multiple unrelated responsibilities, including authentication (`login`), profile management (`update_profile`), admin actions (`ban_user`), product search (`search_products`), and order processing (`place_order`). This creates a monolithic, tightly coupled system where changes to one feature (e.g., modifying ban logic) risk breaking others (e.g., login functionality). The class becomes difficult to maintain, test, and scale, as every modification affects multiple functionalities.

After SRP

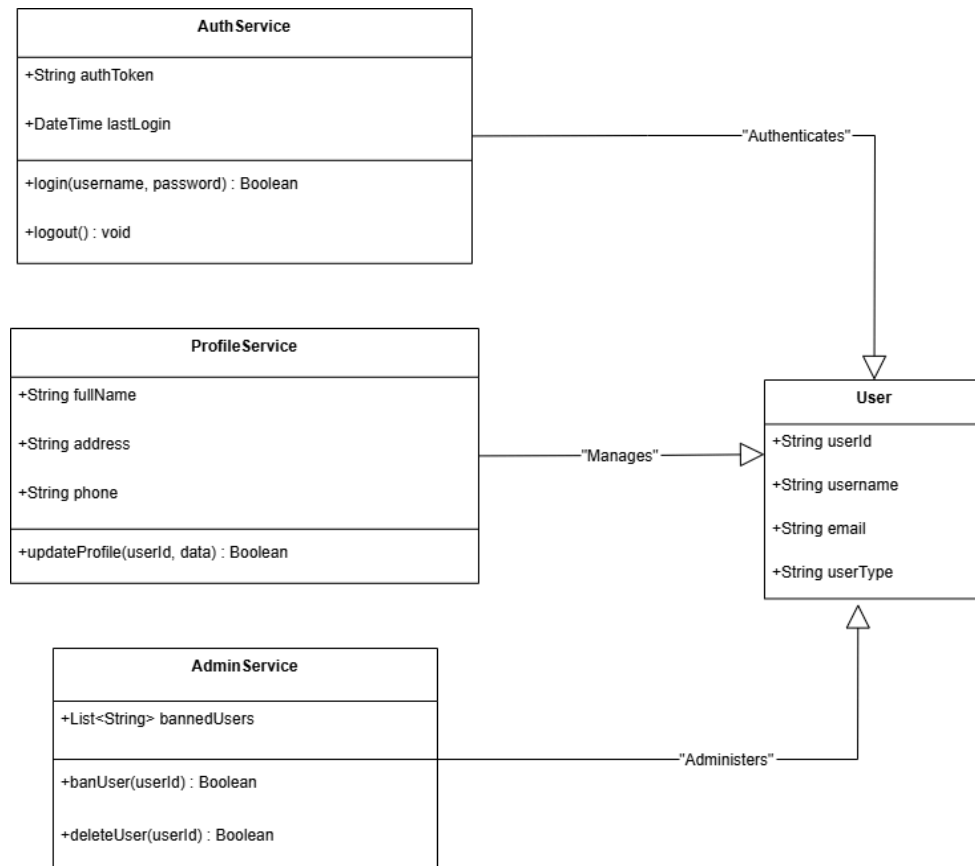


Figure 3.2.1.3 After Single Responsibility Principle (SRP)

```
class AuthService:
|   def login(self): ...

class ProfileService:
|   def update_profile(self): ...

class AdminService:
|   def ban_user(self): ...

class ProductCatalog:
|   def search_products(self): ...
```

Figure 3.2.1.4 Code After Single Responsibility Principle (SRP)

By applying SRP, responsibilities are split into focused, independent classes:

- AuthService handles only authentication (login/logout).
- ProfileService manages only user profile updates.
- AdminService deals only with admin actions (ban/delete users).

Each class now has a single reason to change, reducing complexity and improving maintainability. This modular design aligns with solid principles, making the system more flexible, testable, and scalable.

3.2.2 Open/Closed Principle (OCP)

The Open/Closed Principle (OCP) states that software entities (classes, modules, etc.) should be open for extension (new functionality can be added) but closed for modification (existing code remains unchanged). This is achieved through abstraction (interfaces/abstract classes) and polymorphism, allowing new features to be added without altering stable existing code. In SecondLife, OCP ensures that:

- New report formats (PDF, CSV) can be introduced without changing Report class.

Before OCP

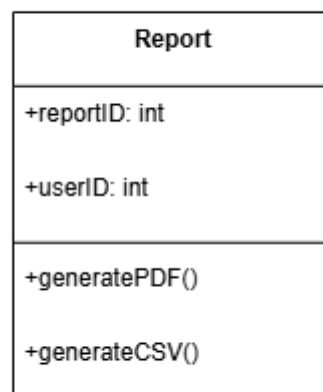


Figure 3.2.2.1 Before Open/Closed Principle (OCP)

```
class Report:
    def generate_pdf(self):
        print("Generating PDF report")

    def generate_csv(self):
        print("Generating CSV report")
```

Figure 3.2.2.2 Before Open/Closed Principle (OCP)

The Report class in its current form violates the Open/Closed Principle (OCP) because it directly implements specific report generation methods (generatePDF() and generateCSV()) within the class. This design forces developers to modify the Report class every time a new report format needs to be added, rather than extending functionality through new classes. For example, adding generateExcel() would require changing the Report class, risking unintended side effects in existing code. This tightly couples report formats to the Report class, making the system less maintainable and scalable.

To adhere to OCP, the Report class should depend on an abstraction (e.g., ReportGenerator interface) and delegate format-specific logic to separate classes (e.g., PDFReport). This way, new formats can be added by creating new classes without modifying the core Report class, keeping it closed for modification but open for extension.

Violation Impact:

- Requires testing the entire Report class for every new format.
- Makes the system resistant to changes (e.g., business requests for new formats).
- Violates the SOLID principle of OCP.

Fix Direction: Replace concrete methods with dependency on an interface (see after-OCP example in previous sections).

After OCP

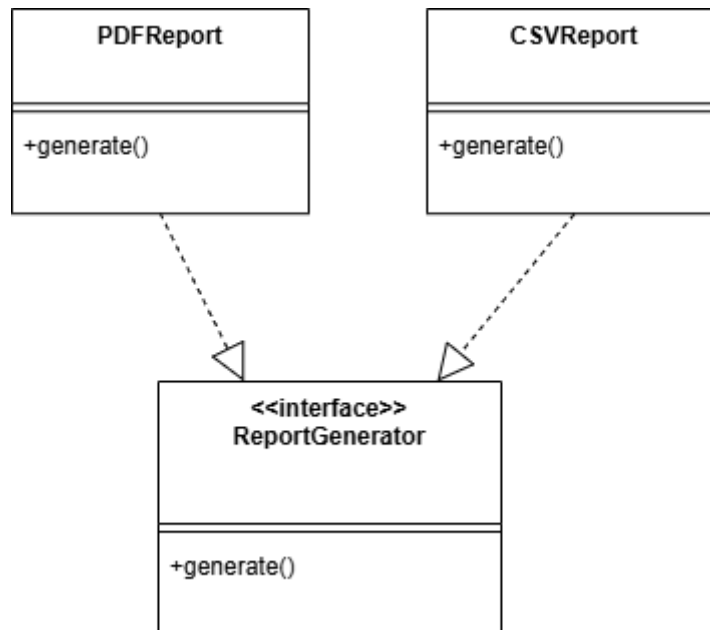


Figure 3.2.2.3 After Open/Closed Principle (OCP)

```
from abc import ABC, abstractmethod

# Report Abstraction
class ReportGenerator(ABC):
    @abstractmethod
    def generate(self): pass

class PDFReport(ReportGenerator):
    def generate(self):
        print("Generating PDF report")

class CSVReport(ReportGenerator):
    def generate(self):
        print("Generating CSV report")

class Report:
    def __init__(self, reportID: int, userID: int):
        self.reportID = reportID
        self.userID = userID

    def generate_report(self, generator: ReportGenerator):
        generator.generate() # Closed for modification
```

Figure 3.2.2.4 After Open/Closed Principle (OCP)

This OCP-compliant design uses a ReportGenerator interface to keep reporting extensible yet stable. Concrete classes like PDFReport and CSVReport implement their own generate() logic, allowing new formats to be added without modifying existing code. The Report class simply calls generate() through the interface, remaining unchanged when extensions are made. This approach minimizes coupling, simplifies testing, and future-proofs the system while following solid principles.

3.2.3 Liskov Substitution Principle (LSP)

The Liskov Substitution Principle (LSP) states that objects of a superclass should be replaceable with objects of its subclasses without breaking the application. In other words, derived classes must:

- Preserve the behavior of the parent class.
- Not strengthen preconditions or weaken postconditions.
- Not throw unexpected exceptions.

For SecondLife, LSP ensures that subclasses (e.g., AdminUser) can seamlessly replace parent classes (e.g., User) without causing errors in order processing, authentication, or other core functionalities.

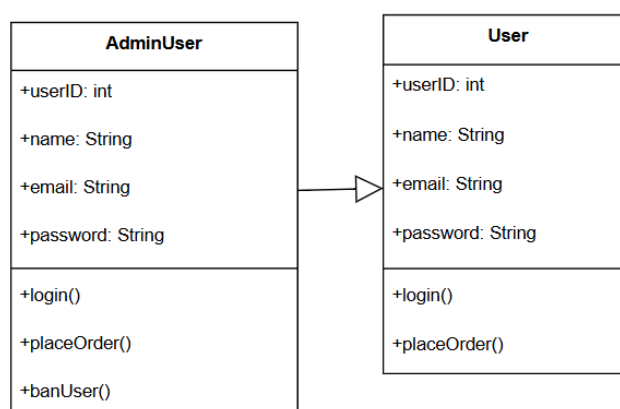


Figure 3.2.2.1 Before Liskov Substitution Principle (LSP)

```

class User:
    def __init__(self, userID: int, name: str, email: str):
        self.userID = userID
        self.name = name
        self.email = email

    def login(self):
        print("User logged in")

    def placeOrder(self):
        print("Order placed")

class AdminUser(User):
    def placeOrder(self):
        raise Exception("Admins cannot place orders") # Violates LSP!

    def banUser(self):
        print("User banned")

```

Figure 3.2.2.2 Before Liskov Substitution Principle (LSP)

Problems Identified

- Violation: AdminUser breaks the behavior of User by throwing an exception in placeOrder().
- Risk: Code expecting User objects fails when substituted with AdminUser.
- Design Flaw: Subclass modifies core functionality instead of extending it.

After LSP

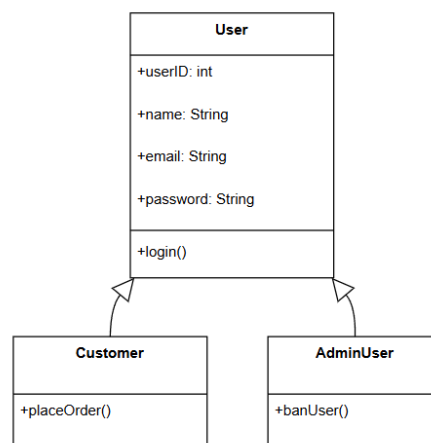


Figure 3.2.2.3 After Liskov Substitution Principle (LSP)

```

from abc import ABC, abstractmethod

class User(ABC):
    def __init__(self, userID: int, name: str, email: str):
        self.userID = userID
        self.name = name
        self.email = email

    @abstractmethod
    def login(self):
        pass

class Customer(User):
    def login(self):
        print("Customer logged in")

    def placeOrder(self):
        print("Order placed") # Valid for customers

class AdminUser(User):
    def login(self):
        print("Admin logged in")

    def banUser(self):
        print("User banned") # Admin-specific method

# Usage:
def process_user(user: User):
    user.login() # Works for both Customer and AdminUser

customer = Customer(1, "Alice", "alice@example.com")
admin = AdminUser(2, "Bob", "bob@example.com")

process_user(customer) # "Customer logged in"
process_user(admin)   # "Admin logged in"

```

Figure 3.2.2.4 After Liskov Substitution Principle (LSP)

Key Improvements

1. Hierarchy Restructuring:
 - Split User into abstract base class with core attributes (userID, email).
 - Customer handles order placement, AdminUser handles admin actions.
2. LSP Compliance:
 - Subclasses preserve User's behavior (e.g., login()).
 - No unexpected exceptions or broken contracts.
3. Extensibility:
 - New roles (e.g., Seller) can extend User without breaking existing code.

3.2.4 Interface Segregation Principle (ISP)

The Interface Segregation Principle (ISP) states that clients should not be forced to depend on interfaces they don't use. Instead of one large interface, we create smaller, focused interfaces so classes only implement what they need. In SecondLife, this means:

- No "fat" interfaces forcing classes to implement irrelevant methods
- Role-specific contracts (e.g., separate interfaces for buyers vs. sellers)
- Reduced coupling between components

Before ISP

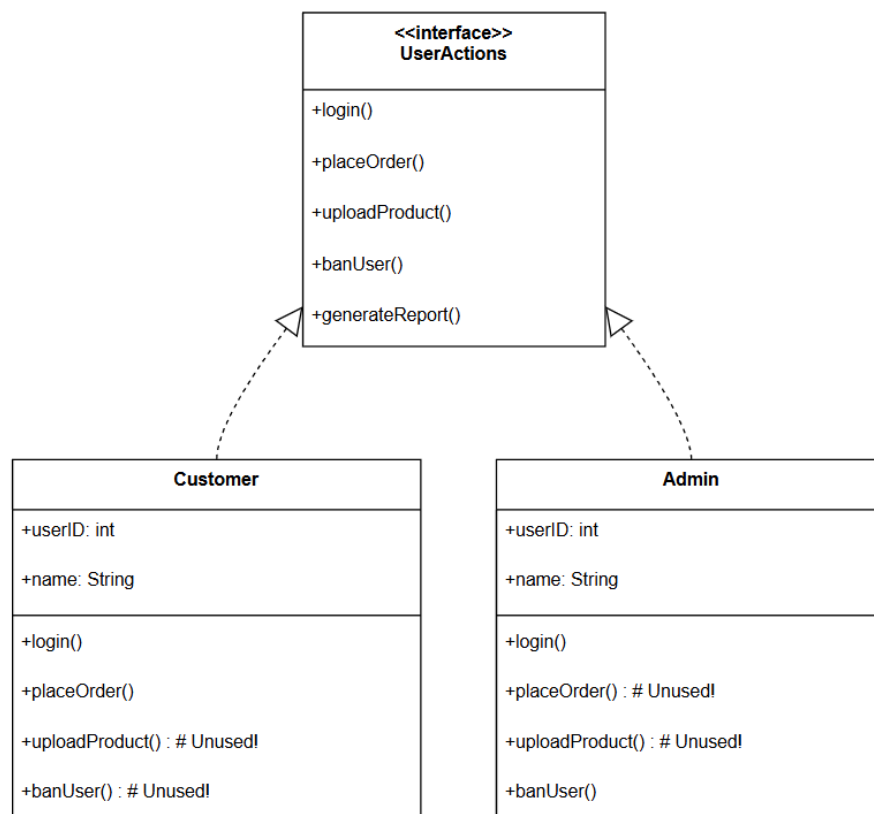


Figure 3.2.4.1 Before Interface Segregation Principle (ISP)

```

from abc import ABC, abstractmethod

class UserActions(ABC):
    @abstractmethod
    def login(self): pass

    @abstractmethod
    def placeOrder(self): pass

    @abstractmethod
    def uploadProduct(self): pass # Irrelevant to customers/admins

    @abstractmethod
    def banUser(self): pass # Irrelevant to customers

class Customer(UserActions):
    def login(self): print("Customer login")

    def placeOrder(self): print("Order placed")

    # Forced to implement unused methods:
    def uploadProduct(self): raise NotImplementedError("Customers can't upload products")

    def banUser(self): raise NotImplementedError("Customers can't ban users")

class Admin(UserActions):
    def login(self): print("Admin login")

    # Forced to implement unused methods:
    def placeOrder(self): raise NotImplementedError("Admins don't place orders")

    def uploadProduct(self): raise NotImplementedError("Admins don't upload products")

    def banUser(self): print("User banned")

```

Figure 3.2.4.2 Before Interface Segregation Principle (ISP)

Problems Identified

- Violation: Customer and Admin are forced to implement irrelevant methods
- Code Smells:
 - NotImplementedError exceptions
 - Methods that will never be called
- Maintenance Issues:
 - Changes to UserActions affect all classes, even if unrelated
 - Difficult to add new user types (e.g., Seller)

After ISP

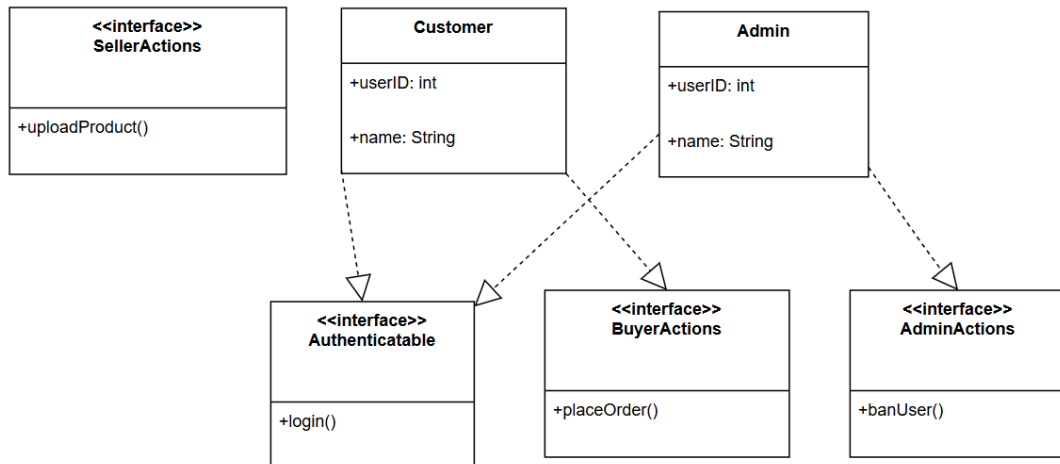


Figure 3.2.4.3 After Interface Segregation Principle (ISP)

```
from abc import ABC, abstractmethod

# Segregated interfaces
class Authenticatable(ABC):
    @abstractmethod
    def login(self): pass

class BuyerActions(ABC):
    @abstractmethod
    def placeOrder(self): pass

class AdminActions(ABC):
    @abstractmethod
    def banUser(self): pass

class SellerActions(ABC):
    @abstractmethod
    def uploadProduct(self): pass

# Implement only needed interfaces
class Customer(Authenticatable, BuyerActions):
    def __init__(self, userID: int, name: str):
        self.userID = userID
        self.name = name

    def login(self): print("Customer login")

    def placeOrder(self): print("Order placed")

class Admin(Authenticatable, AdminActions):
    def __init__(self, userID: int, name: str):
        self.userID = userID
        self.name = name

    def login(self): print("Admin login")

    def banUser(self): print("User banned")
```

Figure 3.2.4.4 After Interface Segregation Principle (ISP)

The refactored design demonstrates proper adherence to the Interface Segregation Principle (ISP) by breaking down the monolithic UserActions interface into smaller, role-specific interfaces (Authenticatable, BuyerActions, AdminActions, SellerActions). Each interface now represents a single capability, allowing classes like Customer, Admin, and Seller to implement only the interfaces relevant to their responsibilities. For example, Customer

implements `Authenticatable` and `BuyerActions` for login and order placement, while `Admin` implements `Authenticatable` and `AdminActions` for login and user management. This eliminates forced implementations of unused methods (like `Customer` needing to stub `banUser()`), reduces coupling between components, and makes the system more maintainable. New roles (e.g., `Moderator`) can now be added by composing only the needed interfaces without affecting existing code. The design also enables type-safe function parameters—a function requiring order placement can explicitly accept `BuyerActions` rather than a generic `User`. This granular approach future-proofs `SecondLife`'s user system, ensuring it remains flexible as new requirements emerge while avoiding interface pollution.

3.2.5 Dependency Inversion Principle (DIP)

The Dependency Inversion Principle (DIP) states that:

1. High-level modules (business logic) should not depend on low-level modules (implementations). Both should depend on abstractions.
2. Abstractions should not depend on details – details should depend on abstractions.

In `SecondLife`, DIP ensures:

- Order processing doesn't depend directly on payment gateways
- Report generation isn't tied to specific formats
- Changes to low-level code (e.g., switching databases) don't require rewriting business logic

Before DIP

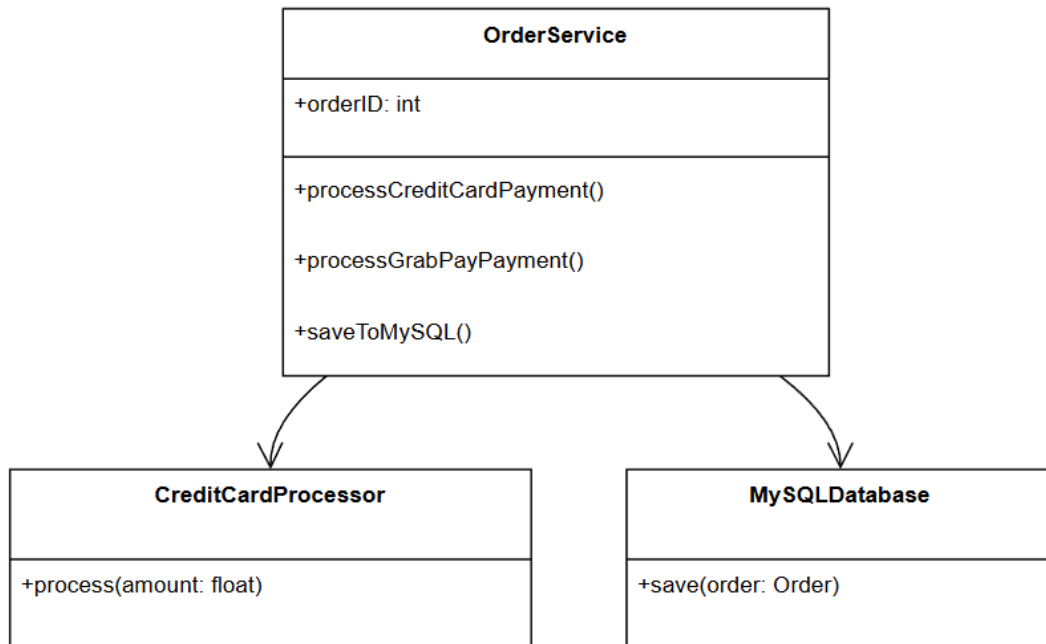


Figure 3.2.4.1 Before Dependency Inversion Principle (DIP)

```
class CreditCardProcessor:
    def process(self, amount: float):
        print(f"Processing credit card payment: ${amount}")

class MySQLDatabase:
    def save(self, order):
        print(f"Saving order {order.id} to MySQL")

class OrderService:
    def __init__(self):
        self.credit_card = CreditCardProcessor() # Direct dependency
        self.db = MySQLDatabase() # Direct dependency

    def process_credit_card_payment(self, amount: float):
        self.credit_card.process(amount)

    def save_to_mysql(self, order):
        self.db.save(order)
```

Figure 3.2.4.2 Before Dependency Inversion Principle (DIP)

Problems Identified

- **Tight Coupling:** OrderService directly depends on concrete implementations
- **Rigidity:** Changing to PayPal or MongoDB requires modifying OrderService
- **Testability:** Hard to mock dependencies for unit tests
- **Violation:** High-level OrderService depends on low-level details

After DIP

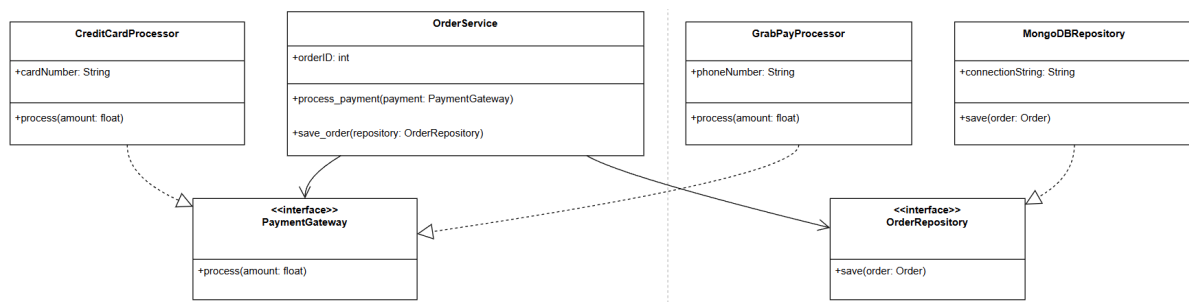


Figure 3.2.4.3 After Dependency Inversion Principle (DIP)

```
from abc import ABC, abstractmethod

# Abstractions
class PaymentGateway(ABC):
    @abstractmethod
    def process(self, amount: float): pass

class OrderRepository(ABC):
    @abstractmethod
    def save(self, order): pass

# Implementations
class CreditCardProcessor(PaymentGateway):
    def __init__(self, card_number: str):
        self.card_number = card_number

    def process(self, amount: float):
        print(f"Processing credit card {self.card_number}: ${amount}")

class GrabPayProcessor(PaymentGateway):
    def __init__(self, phone_number: str):
        self.phone_number = phone_number

    def process(self, amount: float):
        print(f"Processing GrabPay {self.phone_number}: ${amount}")

class MongoDBRepository(OrderRepository):
    def __init__(self, connection_string: str):
        self.connection_string = connection_string

    def save(self, order):
        print(f"Saving to MongoDB: {order.id}")
```

Figure 3.2.4.4 After Dependency Inversion Principle (DIP)

```

# High-level module
class OrderService:
    def __init__(self, payment_gateway: PaymentGateway, repository:
        OrderRepository):
        self.payment_gateway = payment_gateway # Dependency injection
        self.repository = repository

    def process_payment(self, amount: float):
        self.payment_gateway.process(amount)

    def save_order(self, order):
        self.repository.save(order)

# Usage
order_service = OrderService(
    payment_gateway=GrabPayProcessor("+60123456789"),
    repository=MongoDBRepository("mongodb://localhost")
)
order_service.process_payment(100.50)
order_service.save_order(order)

```

Figure 3.2.4.5 After Dependency Inversion Principle (DIP)

The refactored design demonstrates proper adherence to the Dependency Inversion Principle (DIP) through its clear separation of concerns between high-level and low-level modules. By introducing the PaymentGateway and OrderRepository interfaces, the OrderService (high-level business logic) now depends only on abstractions rather than concrete implementations like CreditCardProcessor or MySQLDatabase. This architectural shift enables three key benefits:

- flexibility - payment processors and storage systems can be swapped without modifying core order processing logic
- testability - interfaces allow easy mocking for unit testing business rules without real payment processing
- maintainability - changes to low-level details (like database schema updates) remain isolated from business logic.

The use of dependency injection in OrderService's constructor exemplifies DIP in practice, making dependencies explicit and configurable. This design future-proofs SecondLife's e-commerce system, allowing seamless integration of new payment methods and databases through new interface implementations while keeping the stable order processing rules untouched. The abstraction layer also facilitates team parallelism - frontend and business logic teams can work against interfaces while backend teams implement them.

4. Proposed Design Patterns

4.1 Proposed Design Pattern - Factory Method

4.1.1 Function or Software Component Affected

The Factory Method design pattern is highly effective in situations where a system requires the creation of objects without explicitly specifying their concrete classes at compile time. By delegating the instantiation process to subclasses, this pattern introduces flexibility, allowing the system to adapt dynamically to different object variants. For instance, in applications that need to support multiple product types—such as different UI themes, database connectors, or document formats—the Factory Method enables the definition of a common interface for object creation while letting subclasses determine the exact class to instantiate. This approach not only decouples the client code from concrete implementations but also adheres to the Open/Closed Principle, making it easier to introduce new variants without modifying existing code. As a result, the system becomes more modular, scalable, and maintainable, as object creation logic is centralized and extensible through subclassing.

Affected Functions/Software Components:

1. **Object Creation Logic** – Replaces direct instantiation (`new ConcreteClass()`) with a factory method (`createProduct()`), making the system more extensible.
2. **Dependency Management** – Decouples client code from concrete implementations, adhering to the **Dependency Inversion Principle (DIP)**.
3. **Product Families** – Used in frameworks where derived classes define their own object types (e.g., GUI libraries creating platform-specific buttons).
4. **Testing & Mocking** – Simplifies unit testing by allowing mock objects to be injected via factory methods.

Impact:

- **Maintainability** – New product types can be added without changing existing code.
- **Extensibility** – Subclasses control object creation, supporting Open/Closed Principle (OCP).

- **Reduced Coupling** – Clients depend on abstractions (Document), not concrete classes (PDFDocument).

4.1.2 Description of work flow or data flow

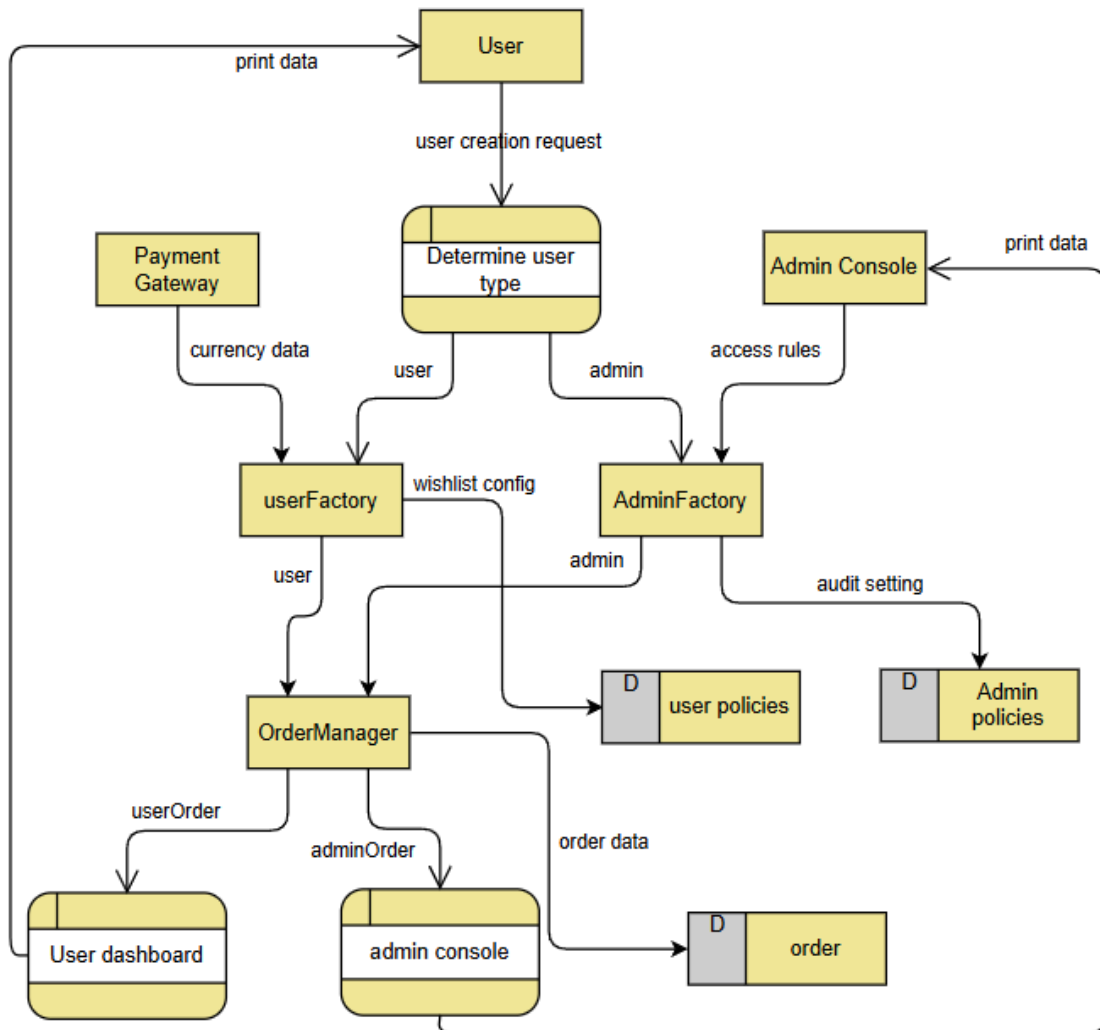


Figure 4.1.2.1 Data Flow Diagram of Factory Method

Data Flow Description

1. Client Request:
 - The Client calls someOperation() on a Creator (e.g., ConcreteCreatorA).
 - The Creator invokes its abstract factoryMethod().

2. Factory Method Execution:

- The ConcreteCreatorA (subclass) implements factoryMethod() and instantiates a ConcreteProductA.
- The Product interface ensures all products (e.g., ConcreteProductA, ConcreteProductB) adhere to the same contract (doStuff()).

3. Product Delivery:

- The created ConcreteProductA is returned to the Creator, which uses it in someOperation().
- The Client interacts with the product via the Product interface, unaware of the concrete class.

Key Points:

- Abstraction: The Client depends only on the Creator and Product interfaces.
- Flexibility: Adding a new product (e.g., ConcreteProductC) requires only a new ConcreteCreator subclass.
- Control Flow: The Creator centralizes object creation logic, while subclasses specialize it.

4.1.3 Sample Class Diagram

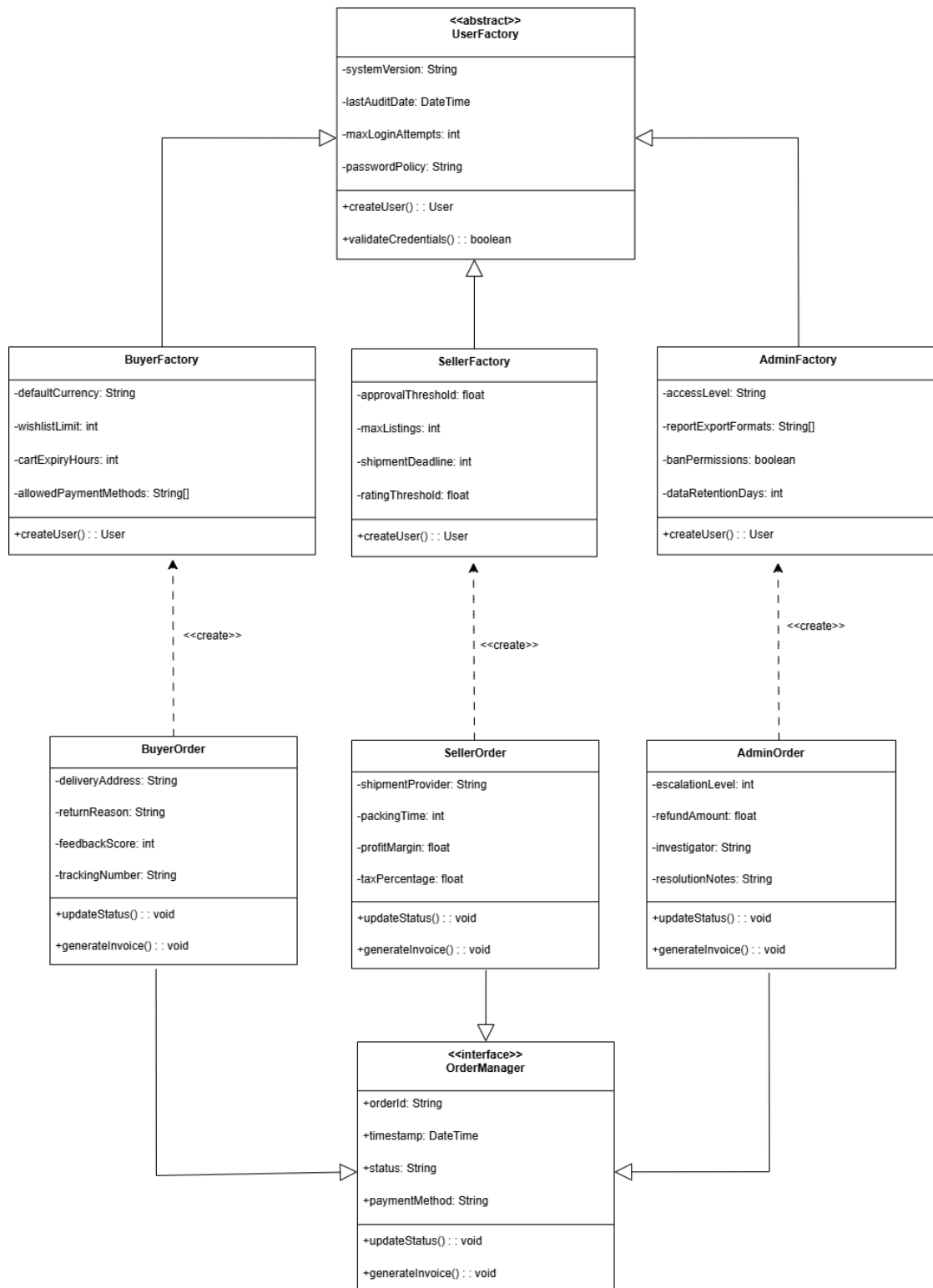


Figure 4.1.3.1 Class Diagram of Factory Method

4.1.4 Sample Other UML Notations (Sequence Diagram)

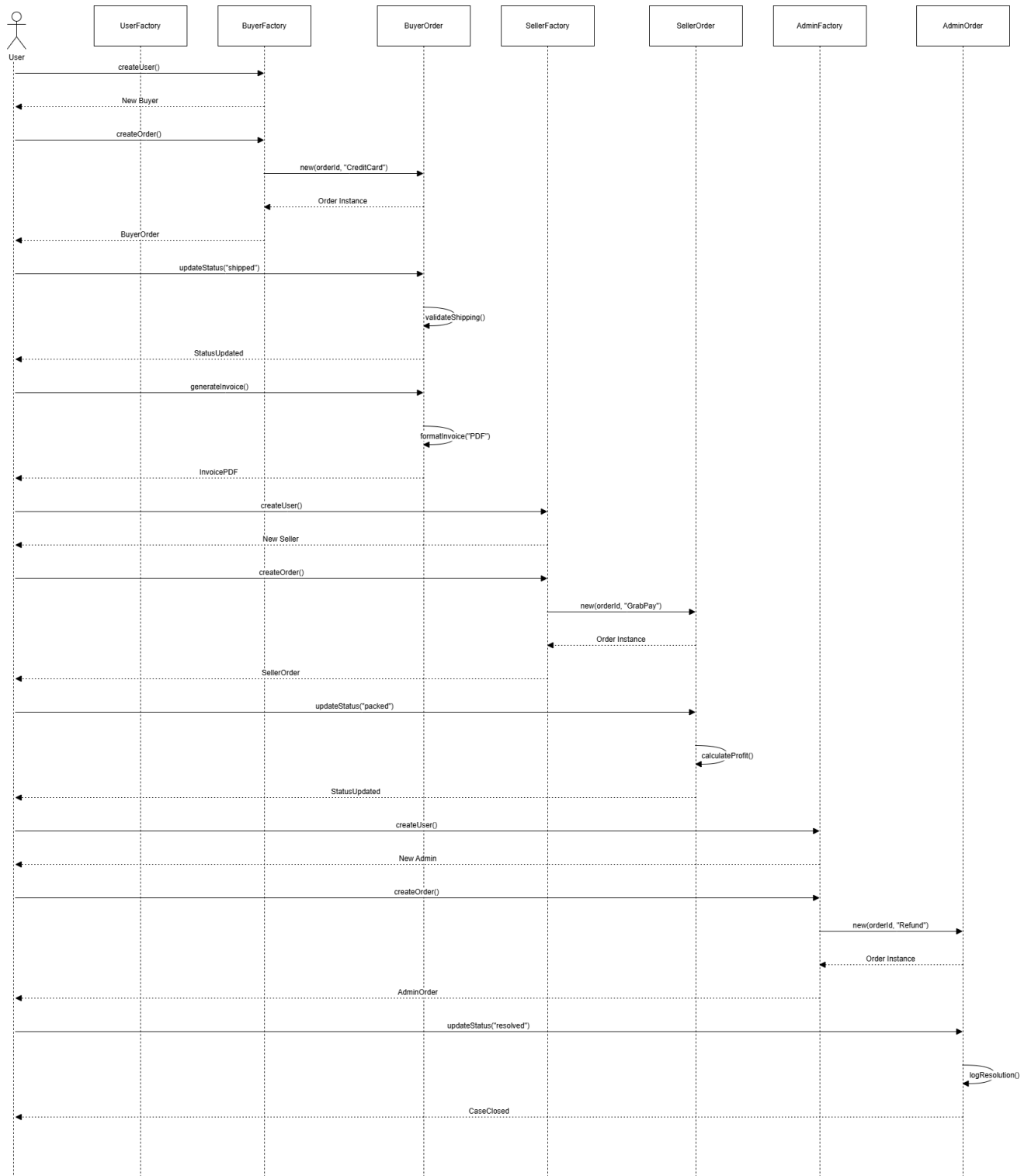


Figure 4.1.3.1 Class Diagram of Factory Method

4.1.5 Sample Potential Code

1. Factory Method Declaration (Abstract Creator)

```
class UserFactory(ABC): # Abstract Creator
    @abstractmethod
    def create_user(self, username, email): # Factory Method
        pass
```

Figure 4.1.5.1 Factory Method Sample Potential Code

- a. This is the core factory method declaration that defines the interface for creating users.
- b. All concrete factories must implement this method.

2. Concrete Factory Implementations

```
class BuyerFactory(UserFactory):
    def create_user(self, username, email): # Factory Method
        Implementation
        return Buyer(username, email, self.default_currency,
self.allowed_payment_methods)

class SellerFactory(UserFactory):
    def create_user(self, username, email): # Factory Method
        Implementation
        return Seller(username, email, self.approval_threshold >
4.0, self.max_listings)

class AdminFactory(UserFactory):
    def create_user(self, username, email): # Factory Method
        Implementation
        return Admin(username, email, self.access_level,
self.ban_permissions)
```

Figure 4.1.5.2 Factory Method Sample Potential Code

- a. Each concrete factory implements its own version of create_user().

- b. They produce different types of objects (Buyer/Seller/Admin) while sharing the same interface.

3. Factory Method Usage in Flask Routes

```
@app.route('/create_user', methods=['POST'])
def create_user():
    user_type = request.json.get('type')

    # Factory Selection
    if user_type == "buyer":
        factory = BuyerFactory()    # Concrete Factory
    elif user_type == "seller":
        factory = SellerFactory()
    elif user_type == "admin":
        factory = AdminFactory()

    # Factory Method Invocation
    user = factory.create_user(username, email)    # << Factory
    # Method Call >>
```

Figure 4.1.5.3 Factory Method Sample Potential Code

- a. The client code doesn't know which concrete factory is being used.
- b. The actual object creation is delegated to the factory method.

4. Order Management (Parallel Implementation)

```
# Interface
class OrderManager(ABC):
    @abstractmethod
    def generate_invoice(self):    # Another factory-like method
        pass

# Concrete Products
class BuyerOrder(OrderManager):
    def generate_invoice(self):    # Implementation for buyers
```

```

        return f"PDF invoice to {self.delivery_address}"

class SellerOrder(OrderManager):
    def generate_invoice(self): # Implementation for sellers
        return "Excel commercial invoice"

```

Figure 4.1.5.4 Factory Method Sample Potential Code

- a. This shows the pattern is also applied to order management.
- b. Each order type implements its own variant of key methods.

4.1.6 Benefit

1. Loose Coupling

- Decouples client code from concrete classes (Buyer, Seller, Admin).
- Example: SecondLife's frontend only interacts with UserFactory interface, not specific user classes.

2. Single Responsibility Principle

- Each factory class handles creation logic for one product type.
- Example: BuyerFactory manages buyer-specific rules (wishlist limits, payment methods).

3. Open/Closed Principle

- New user types can be added without modifying existing code.
- Example: Adding AffiliateUser requires only a new AffiliateFactory subclass.

4. Centralized Object Creation

- Validation logic (e.g., password policies) is enforced in one place.

```

○ class UserFactory(ABC):
○     def validate_credentials(self, password):
○         return len(password) >= 8 # Centralized validation

```

Figure 4.1.6.1 Factory Method Centralized Object Creation Code

5. Polymorphic Behavior

- Factories enable runtime switching of object types.
- Example: Admin dashboard can choose between PDFReport or ExcelReport generators dynamically.

6. Improved Testability

- Mock factories can be injected for unit testing.

```
class MockBuyerFactory(BuyerFactory):  
    def create_user(self):  
        return MockBuyer() # For testing
```

Figure 4.1.6.2 Factory Method Improved Testability Code

4.1.7 Limitations

1. Code Complexity

- Requires creating multiple subclasses, increasing files.
- SecondLife example: Need separate factories for users, orders, reports.

2. Overhead for Simple Cases

- Unnecessary for systems with few object variants.
- Example: If SecondLife only had one user type, a factory would be over-engineering.

3. Tight Coupling Between Products and Creators

- Concrete factories are tied to specific products.
- Example: BuyerFactory can only create Buyer objects, not Seller objects.

4. Refactoring Challenges

- Changing the factory interface requires updating all subclasses.
- Example: Adding a new parameter to create_user() forces changes to all factories.

5. Runtime Type Switching

- Requires client code to know which factory to instantiate.

```
if user_type == "buyer": # Client must decide  
    factory = BuyerFactory()
```

Figure 4.1.7.1 Factory Method Runtime Type Switching Code

6. Limited by Language Features

- Python's dynamic typing reduces some benefits (vs. Java/C# where interfaces are stricter).

4.2 Proposed Design Pattern - Observer

4.2.1 Function or Software Component Affected

The **Observer** design pattern is a behavioral pattern that establishes a one-to-many dependency between objects, allowing multiple observers to automatically react to state changes in a subject they are monitoring. This pattern is particularly useful in event-driven systems where decoupling between the subject and observers is crucial, as it enables dynamic relationships where observers can subscribe or unsubscribe from receiving updates at runtime without modifying the subject's code. The Observer pattern promotes loose coupling by ensuring that the subject has minimal knowledge about its observers, communicating only through a defined interface, which makes the system more modular and easier to extend with new observer types. Common use cases include real-time notifications, event handling in UI frameworks, and monitoring systems where multiple components need to stay synchronized with the state of another object.

The Observer pattern will primarily enhance these SecondLife components:

- Order Status Notifications
 - Affected Components: Order processing system, user dashboards
 - Observation Relationship:
 - Subject: Order status changes (confirmed → shipped → delivered)
 - Observers: Buyers (tracking updates), sellers (inventory updates), admins (audit logs)
- Inventory Management
 - Affected Components: Product listings, seller tools
 - Observation Relationship:
 - Subject: Product stock levels
 - Observers: Buyers (restock alerts), sellers (low-stock warnings)
- Payment Processing
 - Affected Components: Checkout system, accounting
 - Observation Relationship:
 - Subject: Payment status changes (e.g., failed → retry → completed)

- Observers: Buyers (receipts), sellers (sales records), admins (fraud detection)

4.2.2 Description of work flow or data flow

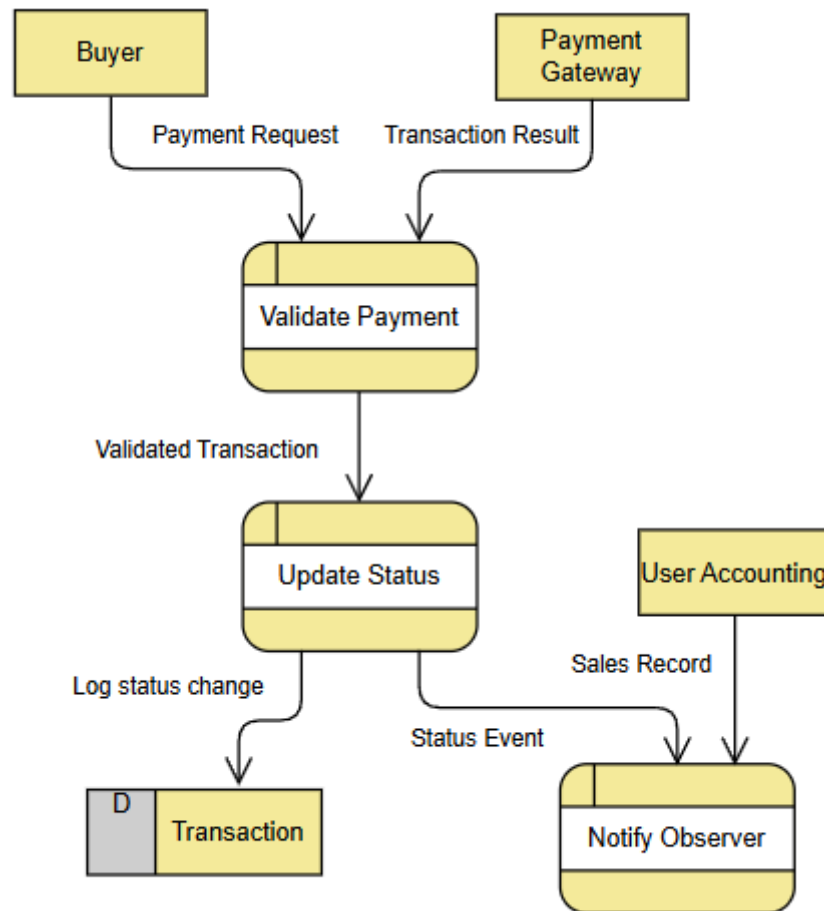


Figure 4.2.2.1 Data Flow of Observer Design Pattern

1. Subject Workflow:

- Validate Payment → Update Status = State change detection.
- Update Status → Transaction DB = Persists state (for recovery).

2. Observer Notification:

- Notify Observer = Broadcasts to all registered observers:
 - Buyers get receipts (Receipt flow).
 - Accounting gets records (Sales Record flow).

3. Observer Pattern Specifics

- **Subject:** Update Status (triggers notifications).
- **Observers:**
 - Buyer: Reacts to receipts.
 - User Accounting: Updates ledgers.
 - (*Implied*) Admin: Monitors for fraud.
- **Loose Coupling:** Observers only know the Status Event interface, not payment internals.

Benefits of Observer

- **Real-Time Updates:** Buyers see payment status instantly.
- **Decoupled Accounting:** Sales records auto-update without polling.
- **Extensibility:** Add new observers (e.g., CRM system) without modifying payment logic.

4.2.3 Sample Class Diagram

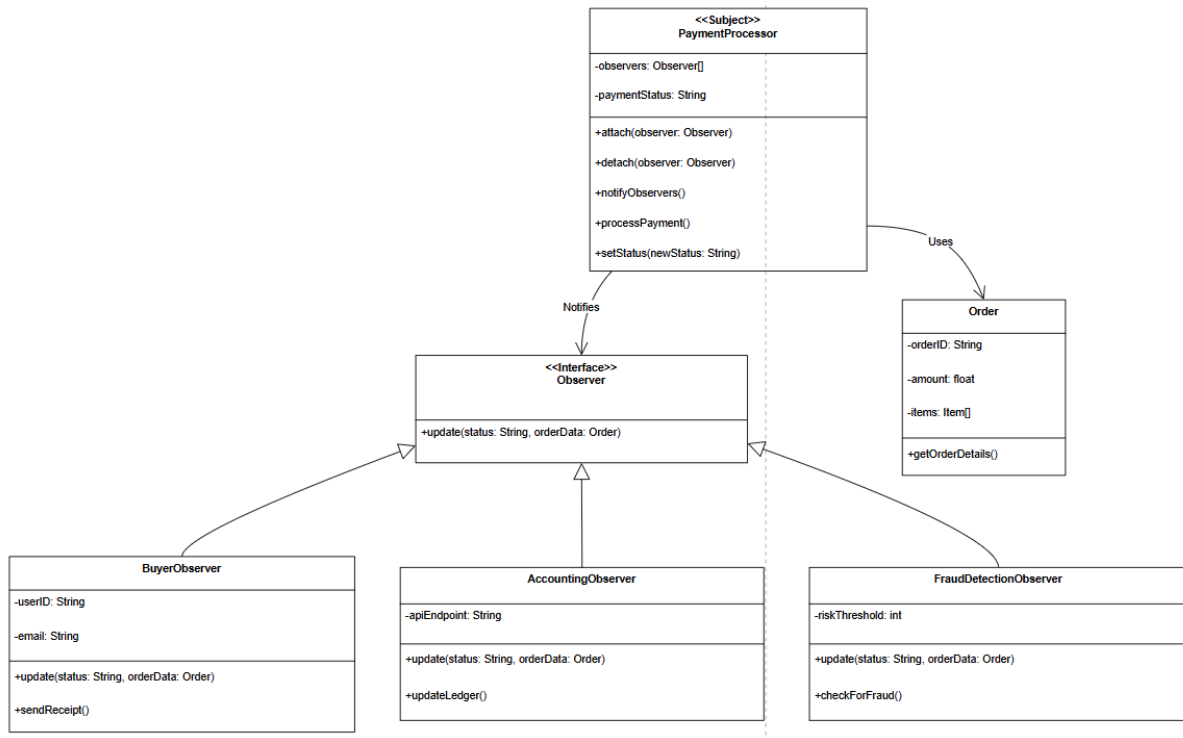


Figure 4.2.3.1 Class Diagram of Observer Design Pattern

4.2.4 Sample Other UML Notations (ex State Diagram, WorkFlow Diagram)

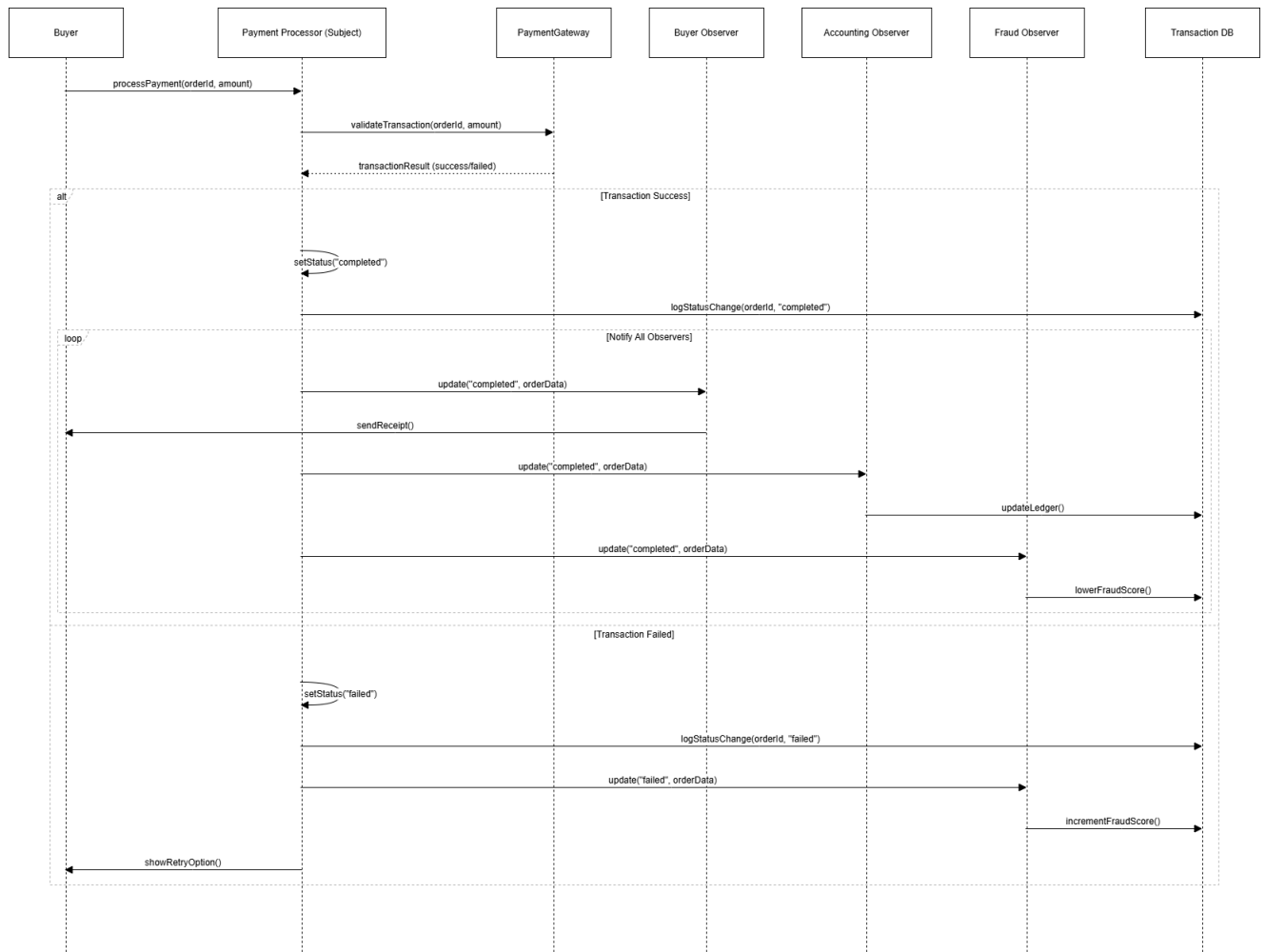


Figure 4.2.4.1 Sequence Diagram of Observer Design Pattern

4.2.5 Sample Potential Code

1. Observer Interface (Core Pattern Structure)

- Defines the contract that all concrete observers must implement
- Ensures uniform notification handling across all observers
- Follows the Open/Closed Principle - new observers can be added without modifying existing code

```
# observer_interface.py
from abc import ABC, abstractmethod

class Observer(ABC): # <<< Observer Pattern Interface
    @abstractmethod
    def update(self, status: str, order_data: 'OrderData'):
        pass
```

Figure 4.2.5.1 Observer Interface Code

2. Subject (PaymentProcessor) - Observer Management

```
class PaymentProcessor:
    def __init__(self):
        self._observers = [] # <<< Observer registry
        self._status = "pending"

    def attach(self, observer: Observer): # <<< Subscription
        if observer not in self._observers:
            self._observers.append(observer)

    def detach(self, observer: Observer): # <<< Unsubscription
        self._observers.remove(observer)

    def notify(self, order_data: OrderData): # <<< Notification
        for observer in self._observers:
            observer.update(self._status, order_data) # <<< Push
update
```

Figure 4.2.5.2 Observer Subject Code

3. Concrete Observers (Notification Handlers)

```
# concrete_observers.py
class BuyerObserver(Observer):
    def update(self, status: str, order_data: OrderData): # <<< Pattern Implementation
        if status == "completed":
            self._send_receipt(order_data) # <<< Observer-specific logic

class AccountingObserver(Observer):
    def update(self, status: str, order_data: OrderData):
        if status == "completed":
            self._update_ledger(order_data) # <<< Business logic

class FraudDetectionObserver(Observer):
    def update(self, status: str, order_data: OrderData):
        if status == "failed":
            self._flag_suspicious(order_data) # <<< Specialized behavior
```

Figure 4.2.5.3 Concrete Observers Code

Observer Workflow:

1. State Change: Payment status updates in PaymentProcessor
2. Notification: notify() calls each observer's update()
3. Custom Handling: Each observer reacts differently:
 - Buyer → Sends receipt
 - Accounting → Updates records
 - Fraud → Checks risk

4. Pattern Initialization (Client Code)

```
processor = PaymentProcessor() # <<< Subject
processor.attach(BuyerObserver()) # <<< Register observer
processor.attach(AccountingObserver())
processor.attach(FraudDetectionObserver())
processor.process_payment(order) # <<< Triggers notifications
```

Figure 4.2.5.4 Observers Pattern Initialization Code

Importance for SecondLife

1. Decoupled Architecture:

- Payment processor doesn't need to know about receipt emails or fraud systems
- Changes to accounting logic don't affect payment processing

2. Real-Time Updates:

- All systems stay synchronized without polling

3. Extensibility:

- New notification channels (e.g., push notifications) can be added without touching payment code

4.2.6 Benefit

1. Loose Coupling

- The Subject (publisher) and Observers (subscribers) are decoupled.
- Changes in one observer don't affect others or the subject.

2. Dynamic Subscription

- Observers can attach/detach at runtime Broadcast Communication
- A single state change can notify multiple systems simultaneously.

3. Open/Closed Principle

- New observers can be added without modifying the subject.

4. Event-Driven Architecture

- Enables real-time updates without polling.

5. Separation of Concerns

- Business logic is isolated from notification logic.

4.2.7 Limitations

1. Uncontrolled Updates

- Subjects don't know if observers fail
- Solution: Use a dead-letter queue for retries.

2. Performance Overhead

- Notifying many observers synchronously can slow down the subject.
 - Solution: Use async calls
3. Memory Leaks
- Observers must explicitly detach, or they linger in memory.
 - Solution: Weak references or automatic cleanup.
4. Debugging Complexity
- Tracking a chain of observer reactions can be difficult.
 - Solution: Log observer executions with correlation IDs.
5. Tight Observer Interface
- All observers must implement `update()`, even if they only care about specific events.
 - Solution: Use event filtering
6. Order Dependence
- Observers may assume they're notified in a specific order.
 - Solution: Explicitly prioritize observers

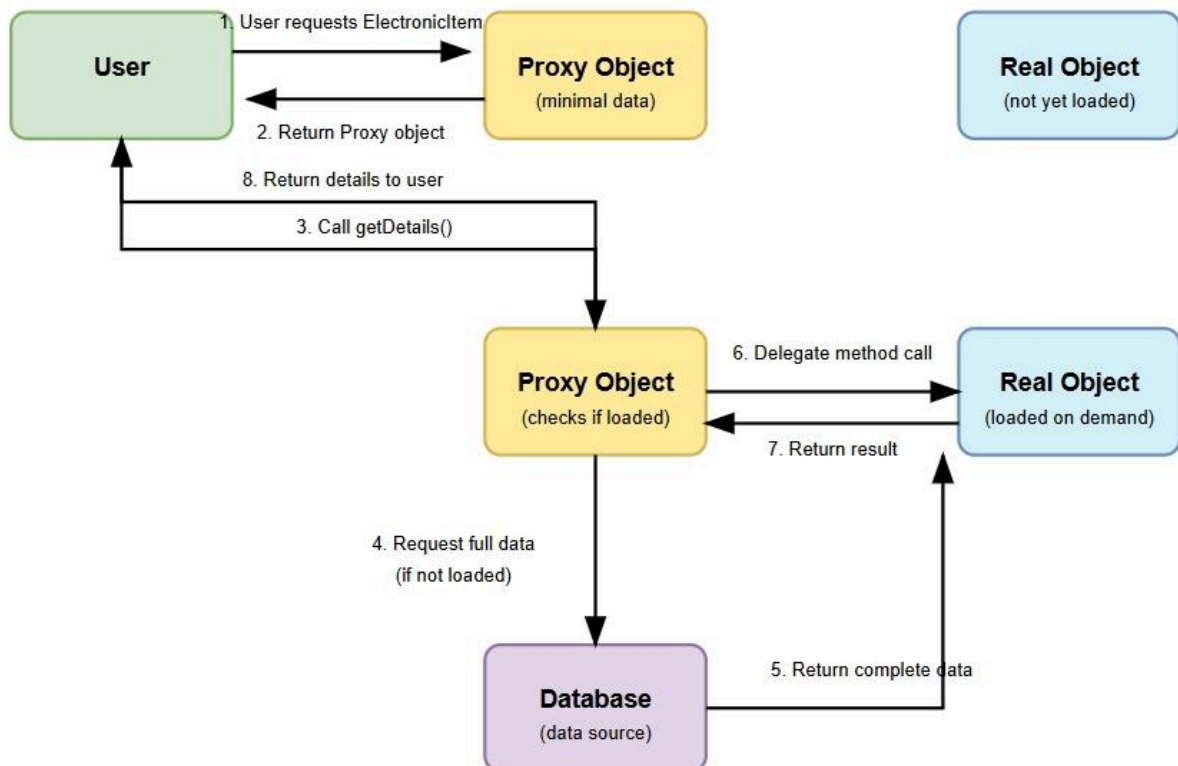
4.3 Proposed Design Pattern - Proxy

4.3.1 Function or Software Component Affected

Component: Electronic Item Viewer / Data Access Layer

Affected Area: Resource-intensive operations like large object loading (e.g., item images, detailed specs, or database fetches)

4.3.2 Description of work flow or data flow



- Client (User Interface) requests access to an ElectronicItem object.
- Instead of directly loading the full object (which might be large or from a remote DB), the Proxy object is returned.
- The Proxy holds minimal data (e.g., name, ID) and defers full loading until a specific method is called (e.g., getDetails()).
- When detailed data is needed, the Proxy retrieves the full object and delegates access to it.

4.3.3 Sample Class Diagram

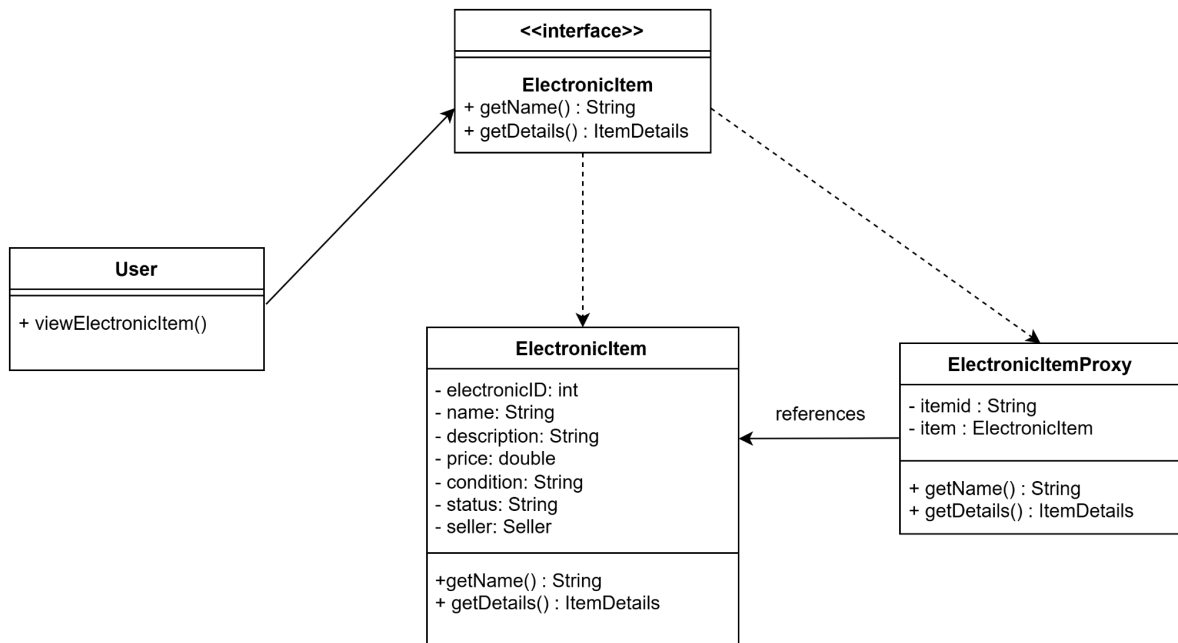


Figure 4.3.1 Sample Class Diagram of Proxy

4.3.4 Sample Other UML Notations (ex State Diagram, WorkFlow Diagram)

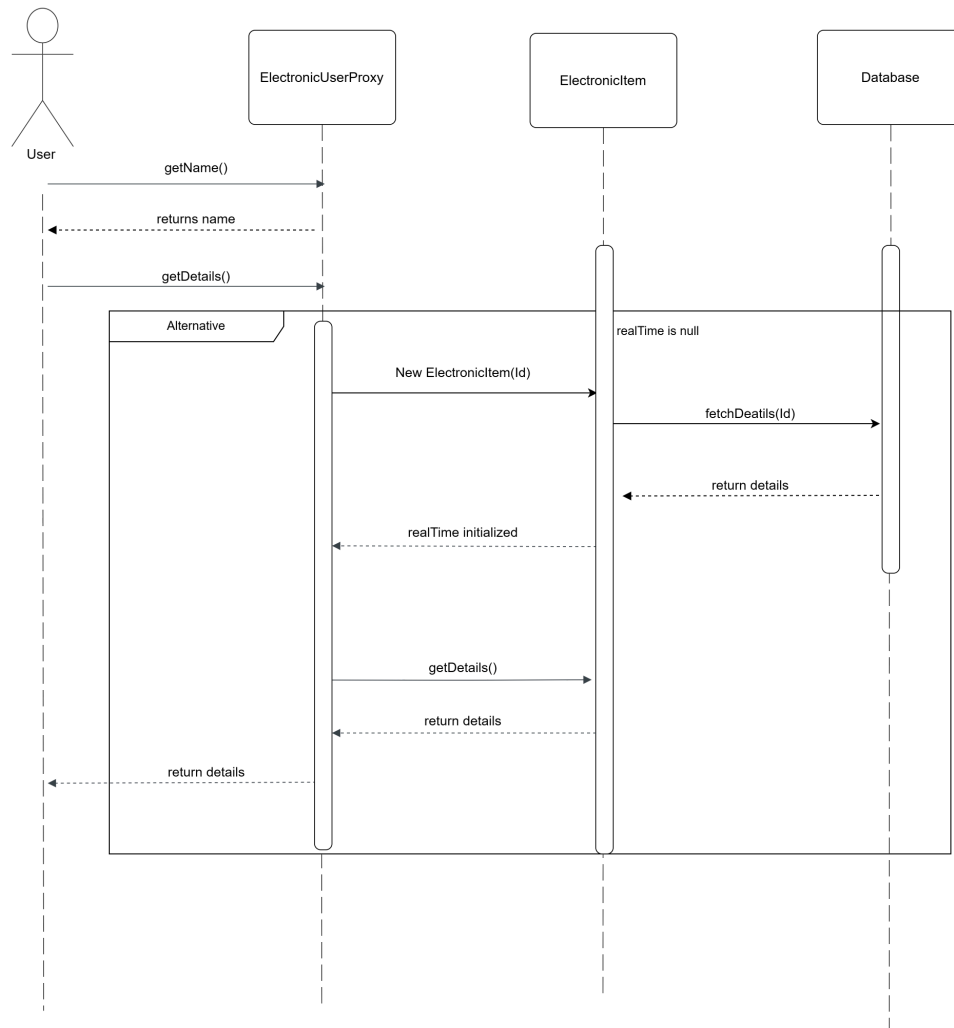


Figure 4.3.2 Sample UML of Proxy

4.3.5 Sample Potential Code

4.3.5.1 Proxy Pattern Implementation

```
import requests

# Define proxy settings
proxies = {
    "http": "http://your_proxy_ip:port",
    "https": "http://your_proxy_ip:port",
}

# Make a request through the proxy
response = requests.get("http://httpbin.org/ip", proxies=proxies)

print("Response via proxy:", response.text)
```

Figure 4.3.3 Proxy Pattern Code (using request with proxy)

4.3.5.2 Proxy Pattern Code (using urllib with a Proxy)

```
import urllib.request

proxy = urllib.request.ProxyHandler({
    "http": "http://your_proxy_ip:port",
    "https": "http://your_proxy_ip:port",
})

opener = urllib.request.build_opener(proxy)
urllib.request.install_opener(opener)

response = urllib.request.urlopen("http://httpbin.org/ip")
print("Response via proxy:", response.read().decode())
```

Figure 4.3.4 Proxy Pattern Code (Using urllib with a Proxy)

4.3.5.3 Proxy Pattern Code (using proxy with authentication)

```
import requests

proxies = {
    "http": "http://username:password@your_proxy_ip:port",
    "https": "http://username:password@your_proxy_ip:port",
}

response = requests.get("http://httpbin.org/ip", proxies=proxies)
print("Response with proxy auth:", response.text)
```

Figure 4.3.5 Proxy Pattern Code (using proxy with authentication)

4.3.6 Benefit

Improved Performance: Delays loading of resource-intensive electronic item data until absolutely necessary, reducing initial load times and memory usage.

Reduced Network Traffic: Only fetches detailed information from the database when specific methods are called, minimizing network traffic for listings and catalog browsing.

Transparency to Client: The client code interacts with the proxy using the same interface as the real object, providing a seamless experience without modifying client code.

4.3.7 Limitations

Added Complexity: Introduces additional classes and indirection that can make the system more complex and harder to understand.

Potential Latency: Users may experience slight delays when accessing detailed information for the first time, as the system needs to load the real data on demand.

Synchronization Issues: In multi-threaded environments, careful synchronization is needed to prevent race conditions when initializing the real object.

4.4 Proposed Design Pattern 4 - Strategy (Payment Functionality)

4.4.1 Function or Software Component Affected

Component:

The Payment Processing System.

Affected Area:

This area is responsible for managing and processing payments within the online portal. The Strategy design pattern is applied to enable dynamic selection of payment methods, error handling, fee calculation, and receipt generation based on the chosen payment method.

4.4.2 Description of work flow or data flow

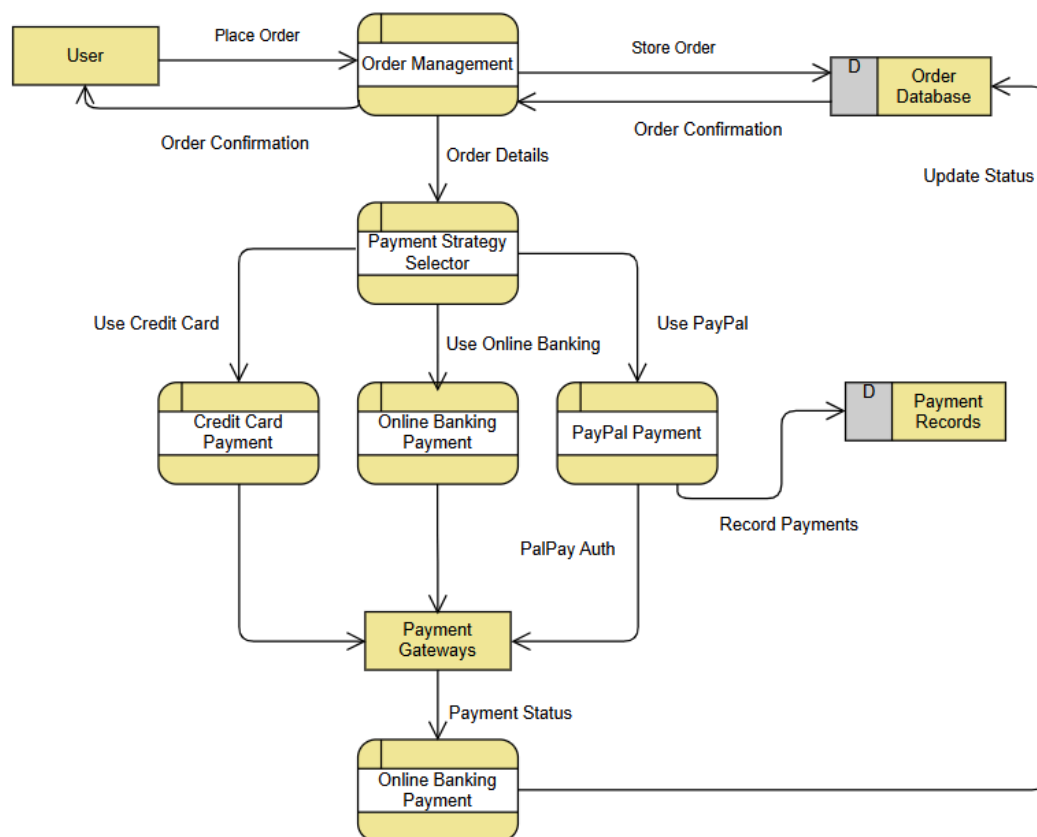


Figure 4.4.1 Strategy Pattern Data-Flow Diagram

1. User places an order and proceeds to the payment page.
2. The system presents multiple payment options.
3. Based on the user's selection, the system dynamically selects the appropriate payment strategy. For example, CreditCardPayment, OnlineBankingPayment, PayPal.
4. The selected strategy processes the payment with relevant APIs or logic.
5. The system receives the payment status and updates the order accordingly.

4.4.3 Sample Class Diagram

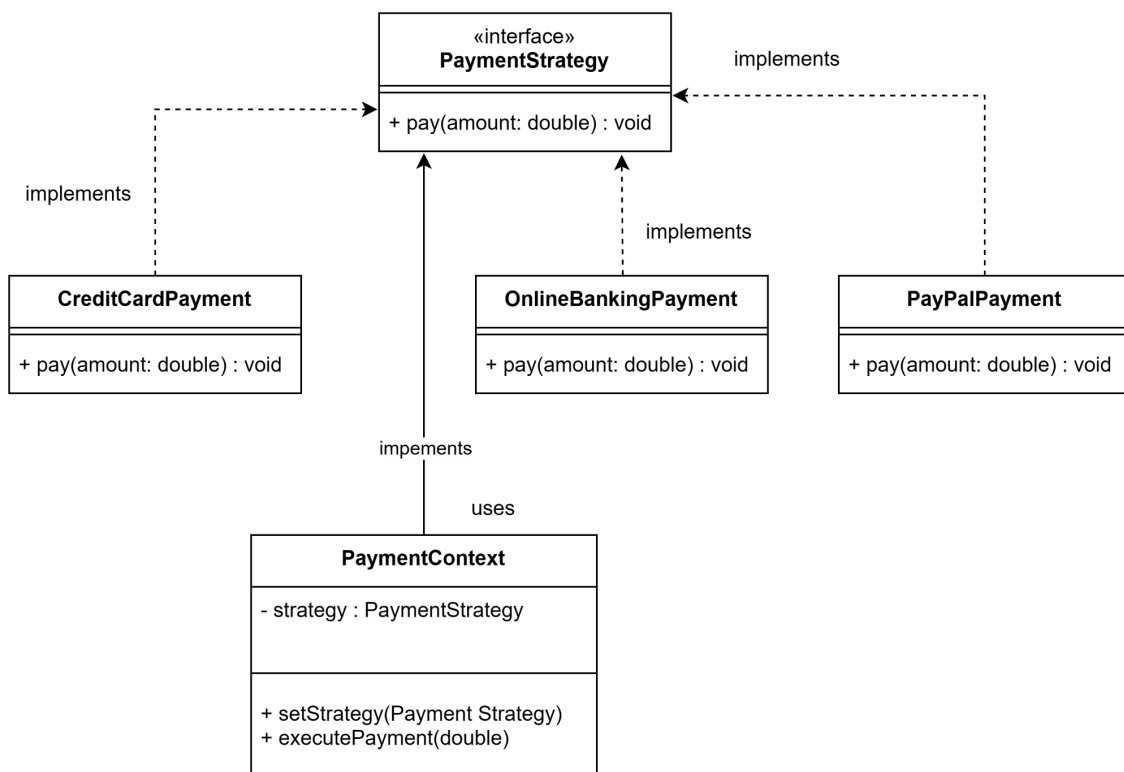


Figure 4.4.2 Strategy Pattern Class Diagram

This diagram illustrates the Strategy design pattern in the payment system:

1. **PaymentStrategy Interface:** Defines the common contract with the pay(amount: double) method that all payment strategies must implement

2. **Concrete Strategy Classes:**
 - **CreditCardPayment:** Implements credit card-specific payment logic
 - **OnlineBankingPayment:** Implements online banking payment processing
 - **PayPalPayment:** Implements PayPal payment handling

3. **PaymentContext:**
 - Contains a reference to the current strategy object
 - Provides setStrategy() to switch payment methods at runtime
 - Uses executePayment() to delegate payment processing to the current strategy

4.4.4 Sample Other UML Notations (ex State Diagram, WorkFlow Diagram)

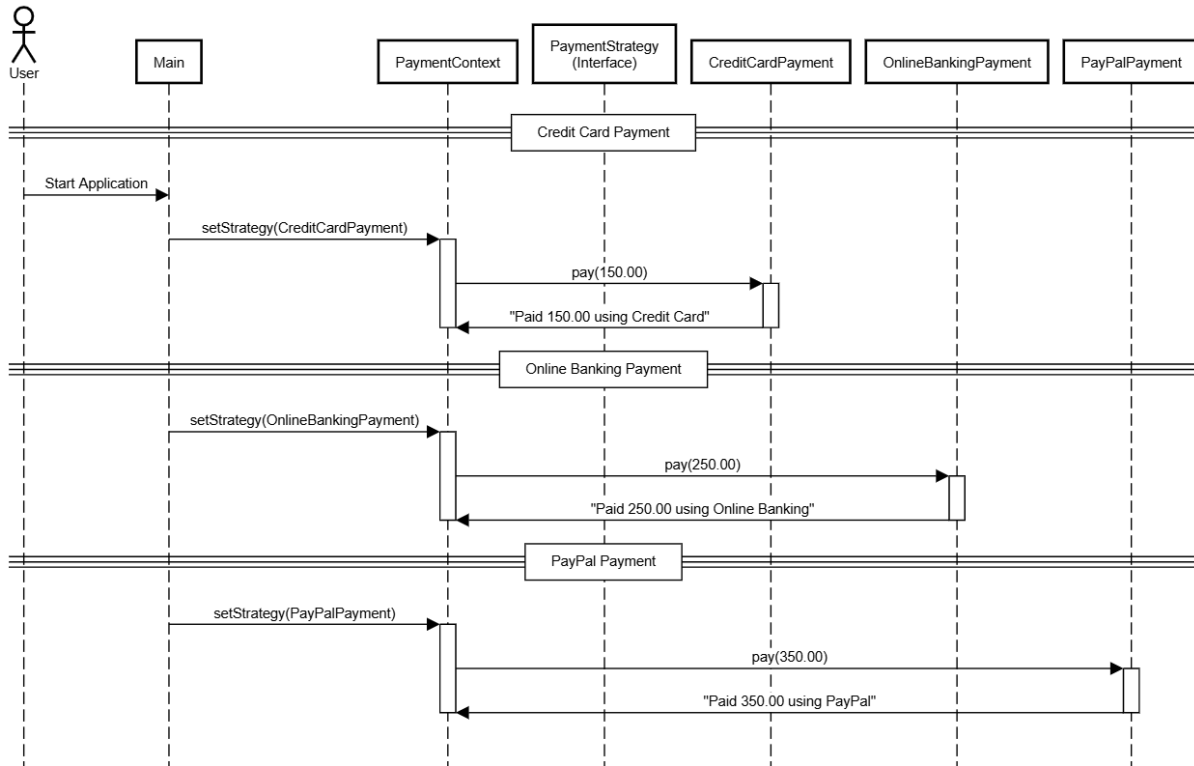


Figure 4.4.3 Strategy Pattern Sequence Diagram

4.4.5 Sample Potential Code

4.4.5.1 PaymentStrategy Interface Implementation

```
from abc import ABC, abstractmethod

# Strategy Interface
class PaymentStrategy(ABC):
    @abstractmethod
    def pay(self, amount):
        pass
```

Figure 4.4.4 Strategy Pattern Code (PaymentStrategy Interface)

4.4.5.2 CreditCardPayment Interface Implementation

```
# Concrete Strategy - Credit Card
class CreditCardPayment(PaymentStrategy):
    def pay(self, amount):
        print(f"[CreditCard] Processing payment of ${amount:.2f}")
        print(f"[CreditCard] Receipt: Paid ${amount:.2f} via Credit Card")
```

Figure 4.4.5 Strategy Pattern Code (CreditCardPayment)

4.4.5.3 OnlineBankingPayment Interface Implementation

```
# Concrete Strategy - Online Banking
class OnlineBankingPayment(PaymentStrategy):
    def pay(self, amount):
        print(f"[OnlineBanking] Processing payment of ${amount:.2f}")
        print(f"[OnlineBanking] Receipt: Paid ${amount:.2f} via Online Banking")
```

Figure 4.4.6 Strategy Pattern Code (OnlineBankingPayment)

4.4.5.4 PayPalPayment Interface Implementation

```
# Concrete Strategy - PayPal
class PayPalPayment(PaymentStrategy):
    def pay(self, amount):
        print(f"[PayPal] Processing payment of ${amount:.2f}")
        print(f"[PayPal] Receipt: Paid ${amount:.2f} via PayPal")
```

Figure 4.4.7 Strategy Pattern Code (PayPalPayment)

4.4.5.5 PaymentContext Interface Implementation

```
# Context Class
class PaymentContext:
    def __init__(self, strategy: PaymentStrategy):
        self.strategy = strategy

    def setStrategy(self, strategy: PaymentStrategy):
        self.strategy = strategy

    def executePayment(self, amount: float):
        try:
            self.strategy.pay(amount)
        except Exception as e:
            print(f"Payment failed: {e}")
```

Figure 4.4.8 Strategy Pattern Code (PaymentContext)

4.4.5.6 Strategy Method Usage in Payment Functionality

```
if __name__ == "__main__":
    amount = 100.00

    # Use Credit Card
    context = PaymentContext(CreditCardPayment())
    context.executePayment(amount)

    print("\n--- Switching to PayPal ---\n")
    context.setStrategy(PayPalPayment())
    context.executePayment(amount)

    print("\n--- Switching to Online Banking ---\n")
    context.setStrategy(OnlineBankingPayment())
    context.executePayment(amount)
```

Figure 4.4.9 Strategy Pattern Code (Usage)

4.4.6 Benefit

(i) Flexible Payment Integration

The Strategy Pattern enables seamless integration of multiple payment methods, such as credit card, online banking, PayPal, without modifying existing system logic. Each payment method is encapsulated within its own class, allowing the system to switch between different strategies at runtime based on user preference or contextual needs.

(ii) Adherence to the Open/Closed Principle

The context class (e.g., PaymentContext) remains unchanged when new payment options are introduced. Developers can add new strategies such as cryptocurrency or e-wallet payments without altering existing code, minimizing the risk of introducing bugs and supporting future scalability.

(iii) Improved Maintainability and Reusability

By isolating each payment behavior into its own class, the system becomes easier to maintain. If a bug is found in one strategy, for example, PayPal processing, it can be fixed in isolation. These modular components are also reusable in other parts of the application or across different projects that require similar payment processing.

4.4.7 Limitations

(i) Increased Class Overhead

Each new payment method requires the creation of a new class that implements the `PaymentStrategy` interface. This can lead to an increased number of classes, which might make the system more complex than necessary, especially for small applications with limited payment options.

(ii) Dependency on Client Awareness

The client or context initializer must be aware of which payment strategy to use and when to switch strategies. Without additional abstraction or logic, this responsibility can introduce tight coupling or decision-making scattered throughout the codebase.

(iii) Potential Overengineering for Simple Scenarios

In applications where only one or two payment methods are needed, implementing the Strategy Pattern may introduce unnecessary abstraction. For such cases, a simple if-else or switch structure might be more efficient and easier to understand.

4.5 Proposed Design Pattern 5 - Facade - Login Functionality

4.5.1 Function or Software Component Affected

The LoginFacade class is the main component affected by the Facade Pattern in the second-hand selling website system.

Login typically involves multiple subsystems such as:

- User input validation
- Database interaction for credential checking
- Session management
- Error handling and feedback to the user

By using the Facade Pattern, we can centralize and simplify the login process through a single interface, improving code readability and maintainability.

The LoginFacade class is responsible for:

- Handling user login requests via a unified method.
- Coordinating between subsystems like user database, authentication logic, and session creation.
- Returning appropriate responses or error messages for both successful and failed login attempts.

4.5.2 Description of Workflow or Data Flow

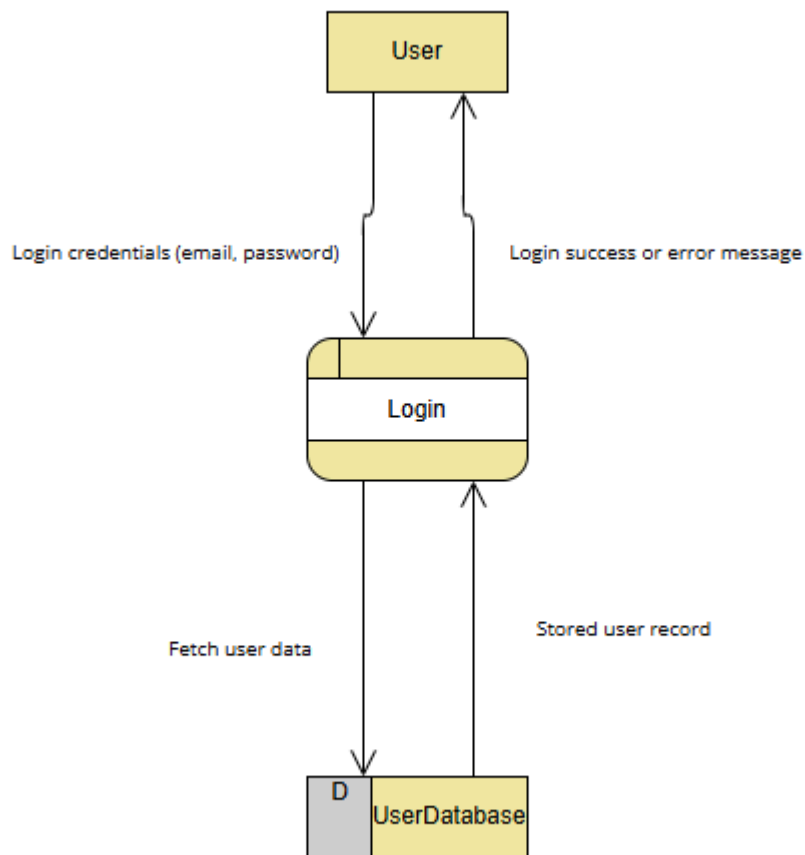


Figure 4.5.2.2 Facade Pattern Level 0 Data-Flow Diagram

The system receives the credentials and passes them to the LoginFacade.

LoginFacade acts as a centralized interface and begins the login process.

It first validates the input format using an internal validation component.

Next, it queries the UserDatabase to fetch the corresponding user record based on the email.

Once the data is retrieved, LoginFacade uses the authentication logic to verify the password against the stored hash.

If the credentials are valid, the LoginFacade creates a session using the session manager.

The user is then granted access and redirected to their dashboard.

If any step fails, the LoginFacade returns a structured error message, prompting the user to retry or recover their password.

4.5.3 Sample Class Diagram

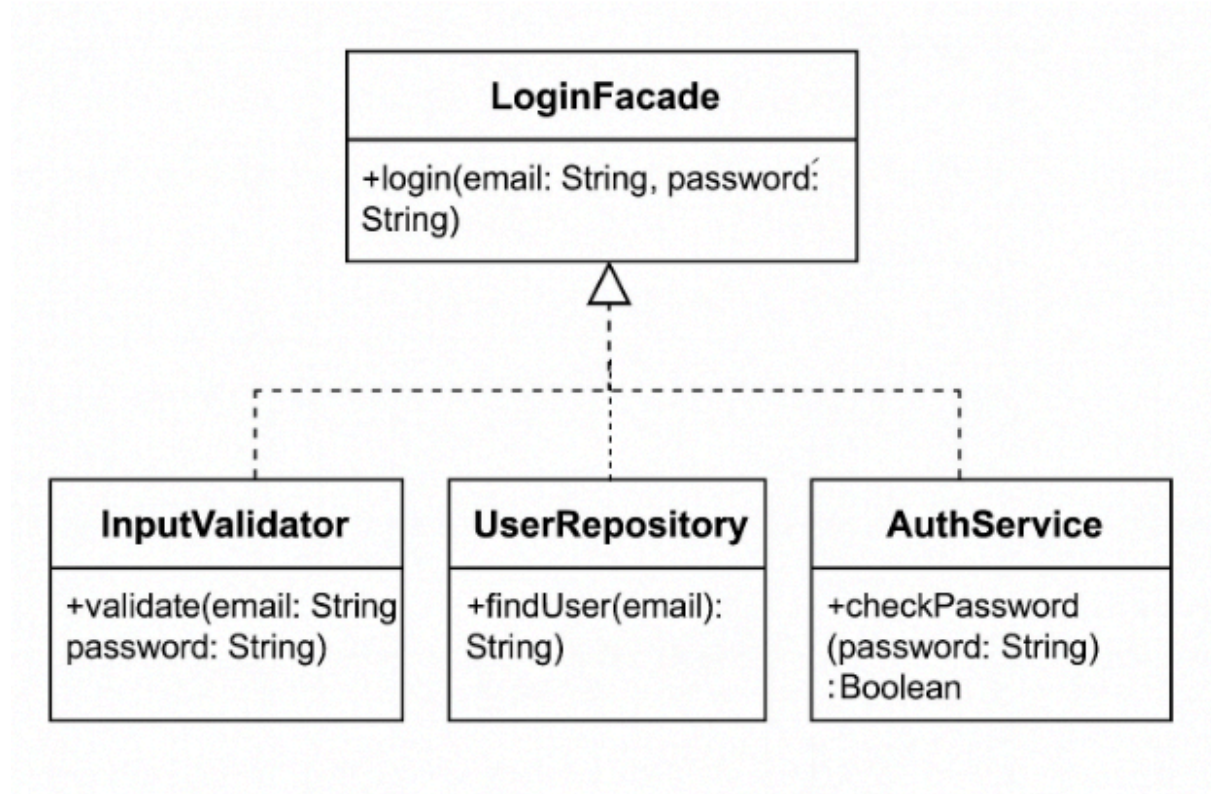


Figure 4.5.3 Facade Pattern Class Diagram

The diagram illustrates how the **Facade Design Pattern** is applied in the login system of the second-hand selling website to centralize and simplify the authentication process.

- **LoginFacade Class (Facade):** Acts as the central access point for the login process. It encapsulates the internal complexities by coordinating multiple subsystems, such as validation, authentication, data access, and session management.
- **InputValidator Class:** Responsible for validating user input, such as checking the format of the email and password, before passing it to other components.

- **UserRepository Class:** Handles interaction with the database. It retrieves stored user information based on the input credentials.
- **AuthService Class:** Verifies the user's password by comparing the input with the hashed password stored in the database.
- **SessionManager Class:** Initiates and manages the user session upon successful login, enabling access to user-specific features.

By applying the Facade Pattern, the login system achieves better modularity, testability, and ease of maintenance. Future enhancements (e.g., two-factor authentication or login analytics) can be implemented behind the LoginFacade without affecting the external interface.

4.5.4 Sample Other UML Notations (sequence diagram)

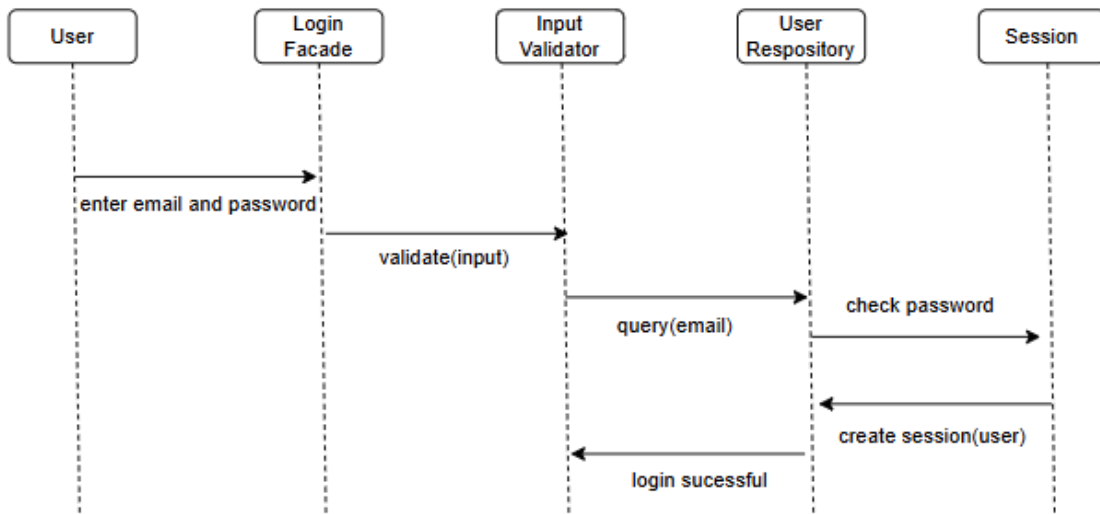


Figure 4.5.4 Facade Pattern Sequence Diagram

4.5.5 Sample Potential Code

Sample Potential Code:

4.5.5.1 Login function using Facade Pattern

```
1  # AuthService.py
2  class AuthService:
3      def verify_credentials(self, email, password):
4          user = UserRepository().get_user_by_email(email)
5          if user and user.password == password:
6              return True
7          return False
8
9  # UserRepository.py
10 class UserRepository:
11     def get_user_by_email(self, email):
12         # Simulated database lookup
13         fake_db = {
14             "user@example.com": "password123"
15         }
16         if email in fake_db:
17             return User(email, fake_db[email])
18         return None
19
20 # InputValidator.py
21 class InputValidator:
22     def is_valid_email(self, email):
23         return "@" in email and "." in email
24
25     def is_valid_password(self, password):
26         return len(password) >= 6
27
28 # SessionManager.py
29 class SessionManager:
30     def create_session(self, email):
31         print(f"Session started for {email}")
32
33 # LoginFacade.py
34 class LoginFacade:
35     def __init__(self):
36         self.validator = InputValidator()
37         self.auth_service = AuthService()
38         self.session_manager = SessionManager()
39
40     def login(self, email, password):
41         if not self.validator.is_valid_email(email):
42             return "Invalid email format."
43         if not self.validator.is_valid_password(password):
44             return "Password must be at least 6 characters."
45         if self.auth_service.verify_credentials(email, password):
46             self.session_manager.create_session(email)
47             return "Login successful."
48         else:
49             return "Invalid email or password."
50
51 # User.py
52 class User:
53     def __init__(self, email, password):
54         self.email = email
55         self.password = password
56
```

Figure 4.5.5.1 Facade Pattern Code (Login)

Usage Example:

4.5.5.2 Facade Pattern Usage in Login Functionality

```
# main.py
facade = LoginFacade()

# Example of a successful login
result = facade.login("user@example.com", "password123")
print(result) # Output: Login successful.

# Example of an invalid login
result = facade.login("user@example.com", "wrongpassword")
print(result) # Output: Invalid email or password.
```

Figure 4.5.5.2 Facade Pattern Code(Login)

4.5.6 Benefits

1. Streamlined User Authentication

The **LoginFacade** centralizes all login-related processes, including input validation, authentication, and session management. This reduces the complexity of the login procedure and makes it easier for developers to maintain and extend the functionality in the future.

2. Improved Maintainability

With the **LoginFacade** acting as the central coordinator for user login, any changes to the login process are isolated within the facade. This makes the overall system easier to maintain because developers only need to modify one class when adjusting login functionality.

3. Seamless Integration with Other Components

The **LoginFacade** simplifies integration with external systems like email verification, third-party authentication (e.g., OAuth), and user management services. It abstracts the complexity, allowing these systems to be plugged into the application

with minimal disruption to the existing workflow.

4. **Enhanced Code Readability**

By using a facade to handle the login process, the system's codebase becomes more readable. Developers working on other parts of the system don't need to dig into the details of how authentication, session management, and validation are implemented. They simply call the facade to handle the process.

4.5.7 Limitations

1. **Potential Overhead for Simple Operations**

While the **LoginFacade** simplifies the process, it may introduce unnecessary complexity for simple login flows. If the application doesn't require advanced login features, using a facade might add more layers than necessary, which can introduce a slight overhead in processing.

2. **Limited Flexibility in Complex Authentication Scenarios**

For more complex authentication scenarios (such as multi-factor authentication or integration with multiple identity providers), the **LoginFacade** may not be flexible enough. Custom authentication flows may require overriding or bypassing the facade, reducing its effectiveness.

3. **Risk of Overloading the Facade**

As more features are added to the login process, there is a risk of the **LoginFacade** becoming too large or handling too many responsibilities. This can lead to a "God Object" scenario, where the facade becomes difficult to maintain, and changes to one part of the login process might inadvertently affect others.

4. **Hidden Complexity**

While the facade simplifies the user interface for developers, it can also hide underlying complexities. This abstraction might make debugging or troubleshooting

more difficult because developers may not be fully aware of the interactions occurring behind the scenes.

5. Conclusion and Suggestion

In conclusion, SecondLife illustrates effectively how design patterns solve real world marketplace problems. The Observer design pattern provides buyer-seller feedback, the Factory Method design pattern provides appropriate verification of products, the Strategy design pattern provides flexible payment options, and lastly the Facade design pattern, which is applied in user authentication. By complying with SOLID principles, the system achieves modularity of design and reduces support overhead while building a specialized platform that addresses trust issues in second-hand electronics trading while promoting sustainable product reuse.

Our suggestions for future development includes a built-in logistics tracking system, which can use the Observer design pattern, an eco-points reward program for incentivizing behaviour, mobile app versions of the SecondLife system for optimal access, and some form of AI price recommendation system. All of these developments would extend SecondLife's current architecture as opposed to requiring significant re-architecting, and still illustrates that key design decisions made at the outset, allows for future development.

Other opportunities for SecondLife to improve accessibility includes providing payment options for cryptocurrency and installment plans, while incorporating social aspects like troubleshooting forums with trending topics into the community, to becoming a haven for electronics enthusiasts. All of these suggestions align with the platform's mission to reduce electronic waste while creating value through a simple and secure marketplace for environmentally conscious technology users who also benefit economically.

6. Bibilography/Reference

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional. ISBN: 978-0201633610. Retrived form <https://www.oreilly.com/library/view/design-patterns-elements/0201633612/>
2. Martin, R. C. (2000). *Design Principles and Design Patterns*. Object Mentor. https://web.archive.org/web/20150906155800/http://www.objectmentor.com/resources/articles/Principles_and_Patterns.pdf
3. Forti, V., Baldé, C.P., Kuehr, R., & Bel, G. (2020). *The Global E-waste Monitor 2020: Quantities, flows and the circular economy potential*. United Nations University. <https://www.itu.int/en/ITU-D/Environment/Pages/Spotlight/Global-Ewaste-Monitor-2020.aspx>
4. Carousell. (2024). Buy and sell new & secondhand items. Retrieved from <https://www.carousell.com.my/>
5. Mudah.my. (2024). Malaysia's largest marketplace. Retrieved from <https://www.mudah.my/>
6. Facebook Marketplace. (2024). Buy and sell items locally or have something shipped. Retrieved from <https://www.facebook.com/marketplace/>